

Zadanie 2: optymalizacja z ograniczeniami

Adam Tokarz, Serhii Zeliuk

EAIIB ISI Metody i algorytmy optymalizacji grupa laboratoryjna 2 poniedziałek 18:30

Termin realizacji: 31 marca 2025

Wybierz funkcję testową wykorzystaną w zadaniu 1.

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Dodaj ograniczenie postaci $x_1^2 + x_2 + b = 0$ ze stałą b dopasowaną w taki sposób, aby żadne minimum lokalne (przynajmniej w zakresie w którym losowany jest punkt początkowy) nie spełniają ograniczenia.
2. Zaimplementuj metodę funkcji kary do rozwiązania tego problemu.
3. Wylosuj 10 punktów z dziedziny przeszukiwania z tabelki. Dla każdego z nich przeprowadź 100 kroków optymalizacji metodą największego spadku ze stałym krokiem. Narysuj wykres zależności wartości funkcji optymalizowanej od kroku optymalizacji.
4. Przeprowadź procedurę dla kilku kroków. Spróbuj zilustrować brak zbieżności, szybką zbieżność i powolną zbieżność.

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Zamień metodę największego spadku na metodę gradientów sprzężonych.

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Wykonaj benchmarking metody z użyciem `BenchmarkTools.jl`. Zanotuj czasy działania wywołań optymalizacji oraz liczbę alokacji. Spróbuj zoptymalizować działanie funkcji korzystając wymienionych tu rad: [Julia performance tips](#). W

sprawozdaniu napisz jakie zmiany wykonane i jak wpłynęły na czas działania programu.

```
In [112... using LinearAlgebra
using Plots
using Random
```

1.1 Funkcja z zad.1

Three-hump camel function

f(vec) - Funkcja zwraca wynik funkcji Three-hump camel function na podstawie podanego wektora argumentów. Tą funkcję będziemy chcieli zoptymalizować.

f_grad(vec) - Funkcja oblicza gradient, pochodne cząstkowe funkcji w dwóch kierunkach, zwraca listę dwuelementową, pierwszy element to pochodna cząstkowa po x, drugi po y

```
In [113... function f(vec)
    x, y = vec[1], vec[2]
    return 2 * x^2 - 1.05 * x^4 + x^6 / 6 + x*y + y^2
end

function f_grad(vec)
    x, y = vec[1], vec[2]
    return [4 * x - 4.2 * x^3 + x^5 + y, x + 2 * y]
end
```

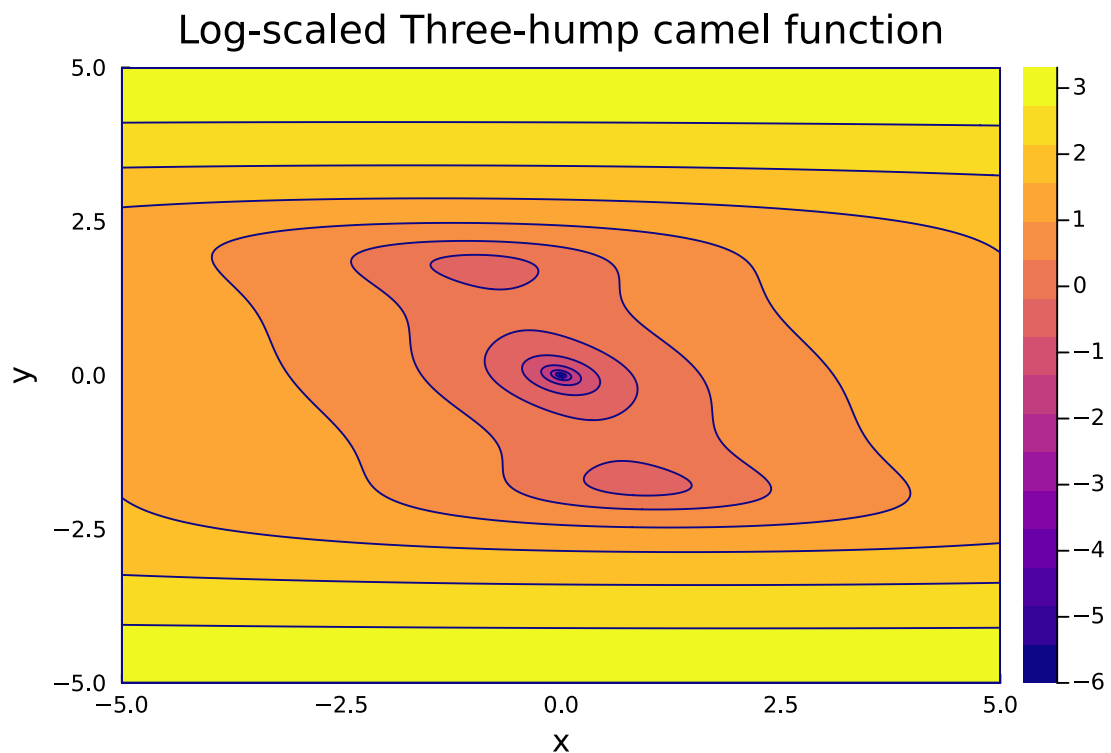
f_grad (generic function with 1 method)

1.2 Wizualizacja funkcji

Obliczanie wartości funkcji dla wartości x i y w przedziale [-5, 5]. Wartości na wykresie skalowane są logarytmicznie w celu lepszej wizualizacji.

```
In [114... x = -5:0.01:5
y = -5:0.01:5
z = zeros(length(x), length(y))
for i in 1:length(x)
    for j in 1:length(y)
        z[i, j] = f([x[i], y[j]])
    end
end

z_log = log10.(abs.(z) .+ 1e-6)
contourf(x, y, z_log, fill=true, color=:plasma, xlabel="x", ylabel="y", title="L
```



Wykres 1 Wykres wartości funkcji f - widoczne na nim są 3 minima lokalne, środkowe minimum jest minimum globalnym. Wartości skalowane logarytmicznie.

1.3 Losowanie punktów

`rand_uniform(a, b)` - Funkcja losująca wartości z przedziału $[a, b]$

`points` - lista z 10 wylosowanymi punktami z przedziału $[-5, 5]$ dla x i y

In [115...

```
function rand_uniform(a, b)
    return rand() * (b-a) + a
end

# lista z 10 wylosowanymi punktami
points = [[rand_uniform(-5, 5), rand_uniform(-5, 5)] for i in 1:10]
points
```

```
10-element Vector{Vector{Float64}}:
 [3.2591126259441854, -4.400799746868519]
 [-4.8734496769236655, 3.821377482653876]
 [1.1215535295861931, 4.093881144666257]
 [3.88687225409787, -1.1275735812556453]
 [-0.25874721309296334, -3.822494792506128]
 [1.9537136594137392, 4.148265872613166]
 [3.024558630625405, -4.114926588718121]
 [-2.522681255057994, 0.5129298119364822]
 [-1.5428496448177311, -2.2034718854456194]
 [-4.839463349215785, -1.8722666120552756]
```

1.4 Funkcje z ograniczeniami

Poniżej znajdują się funkcje, które zostaną zastosowane w optymalizacji z ograniczeniami:

- **g(vec)** - funkcja ograniczenia g w postaci paraboli 3D, przyjmuje listę z 2 wartościami i zwraca wyliczoną wartość. Wzór: $g(x, y) = x^2 + y + b$. Parametr $b = 1.5$ został dobrany tak, aby ograniczenie nie przechodziło przez żadne z minimów lokalnych.
- **g_grad(vec)** - funkcja obliczająca pochodne cząstkowe po x i po y funkcji ograniczenia g, przyjmuje listę 2 wartości i zwraca listę pochodnych cząstkowych.
- **p(vec)** - funkcja kary p, jest to wartość funkcji g podniesiona do kwadratu
- **p_grad(vec)** - wylicza pochodną z funkcji p
- **fp(vec, p)** - funkcja kary fp, stosowana do obliczania wartości kary w zależności od wartości funkcji f. Parametry: vec - punkt o wartości x i y, p - parametr przy funkcji kary. Zwraca: wyliczoną wartość liczbową.
- **fp_grad(vec, p)** - gradient funkcji fp.

In [116...

```
function g(vec)
    x, y = vec[1], vec[2]
    return x^2 + y + 1.5
end

function g_grad(vec)
    x, y = vec[1], vec[2]
    return [2 * x, 1]
end

function p(vec)
    return g(vec)^2
end

function p_grad(vec)
    return 2 * g(vec) * g_grad(vec)
end

function fp(vec, p)
    return f(vec) + p * p(vec)
end

function fp_grad(vec, p)
    return f_grad(vec) + p * p_grad(vec)
end
```

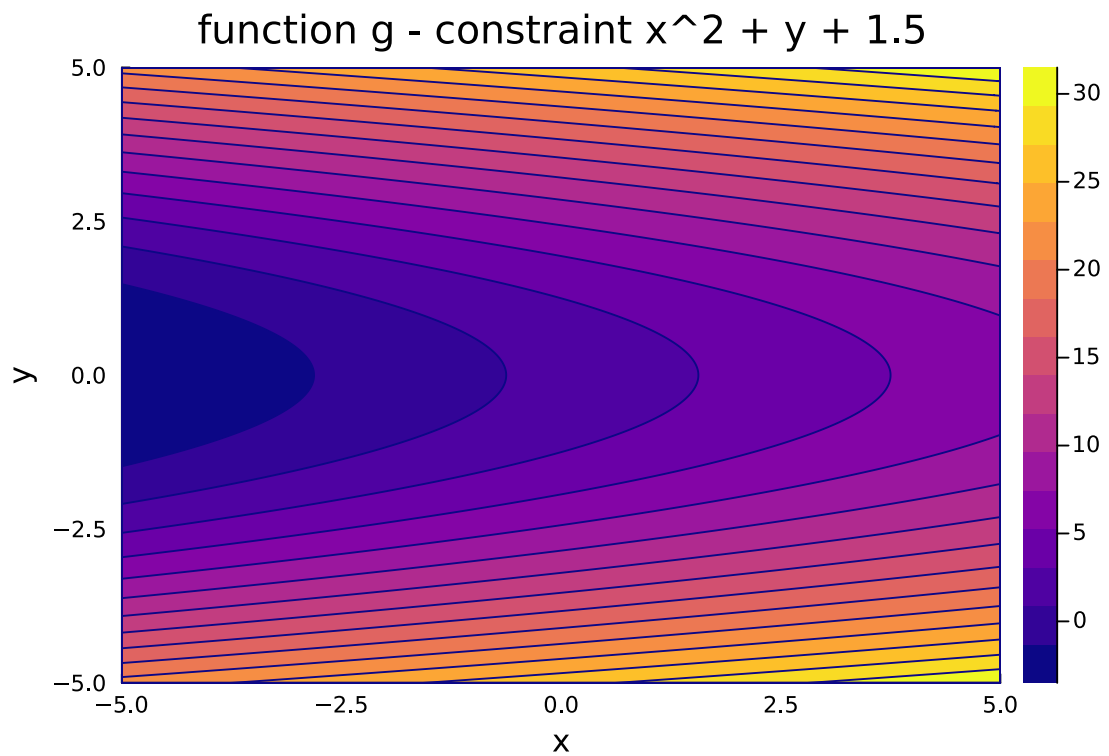
fp_grad (generic function with 1 method)

1.5 Wizualizacja ograniczenia

In [117...

```
z1 = zeros(length(x), length(y))
for i in 1:length(x)
    for j in 1:length(y)
        z1[i, j] = g([x[i], y[j]])
    end
end

contourf(x, y, z1, fill=true, color=:plasma, xlabel="x", ylabel="y", title="func
```



Wykres 2 Wykres wartości funkcji ograniczenia g, wartości x i y w przedziale [-5, 5].
Widoczny jest charakterystyczny paraboliczny kształt.

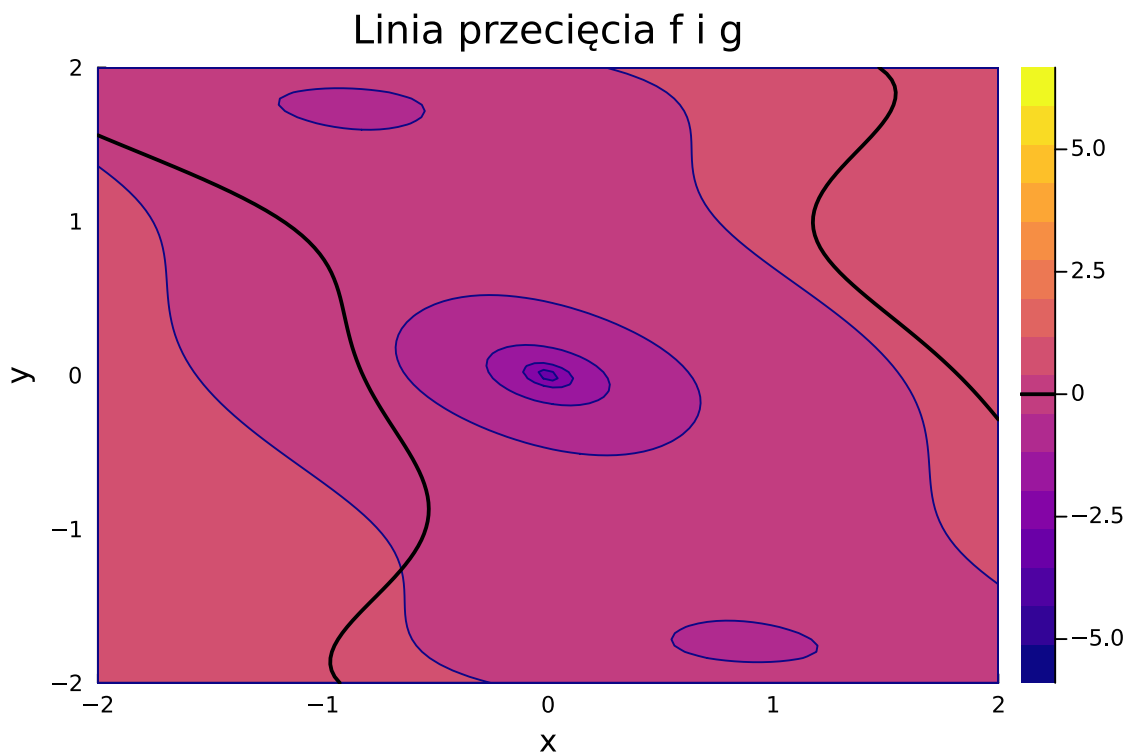
```
In [118... x_range = LinRange(-2, 2, 100)
y_range = LinRange(-2, 2, 100)

z_f = [f([x, y]) for x in x_range, y in y_range]
z_g = [g([x, y]) for x in x_range, y in y_range]

z_diff = z_f - z_g

z_log = log10.(abs.(z_f) .+ 1e-6)

contourf(x_range, y_range, z_log, fill=true, color=:plasma, xlabel="x", ylabel="y")
Plots.contour!(x_range, y_range, z_diff, levels=[0], color=:black, linewidth=2,
```



Wykres 3 Wykres funkcji f z czarnymi liniami będącymi zaznaczonym przecięciem wartości funkcji f z funkcją g .

1.6 Metoda największego spadku ze stałym krokiem - funkcja kary, optymalizacja z ograniczeniem.

- **`gradient_descent_with_penalty_method(fx, fx_grad, px, px_grad, fpx, fpx_grad, x0, k_max,  $\alpha$ ,  $\rho=1.0$ ,  $\gamma=2.0$ , atol=1e-8, max_iter=100, tol=1e-6)`** - Funkcja implementująca algorytm metody największego spadku wraz z funkcją kary. Zwraca znalezione minimum oraz wartości funkcji f i p dla kolejnych iteracji.

Argumenty:

- fx - funkcja optymalizowana
- fx_grad - gradient funkcji optymalizowanej
- px - funkcja kary
- px_grad - gradient funkcji kary
- fpx - funkcja kary w zależności od wartości funkcji optymalizowanej
- fpx_grad - gradient funkcji fpx
- $x0$ - punkt początkowy, od którego zaczynamy optymalizację
- k_max - maksymalna liczba kroków dla ograniczenia
- α - wartość kroku, przez który mnożony jest gradient w metodzie największego spadku
- $\rho=1.0$ - początkowa wartość parametru przy funkcji kary
- $\gamma=2.0$ - parametr zwiększający parametr ρ w kolejnych iteracjach
- $atol=1e-8$ - współczynnik tolerancji dla metody kary

- max_iter=100 - maksymalna liczba iteracji dla metody optymalizacji największego spadku
- tol=1e-6 - współczynnik tolerancji dla metody największego spadku

In [119...

```
function gradient_descent_with_penalty_method(fx, fx_grad, px, px_grad, fpx, fpx_grad)
    x = x0
    theta = copy(x0)
    f_values = []
    p_values = []
    storage = zeros(length(theta))
    for k in 1 : k_max
        for i in 1:max_iter
            value_start = fpx(theta, p)
            storage = fpx_grad(theta, p)
            norm_storage = norm(storage)
            theta_new = theta - storage .* (alpha / norm_storage)
            value_stop = fpx(theta_new, p)
            push!(f_values, fx(theta))
            push!(p_values, px(theta))
            if abs(value_stop - value_start) < tol
                break
            else
                theta = theta_new
            end
        end
        x = theta
        p *= gamma
        if p(x) < atol
            return x, f_values, p_values
        end
    end
    return x, f_values, p_values
end
```

gradient_descent_with_penalty_method (generic function with 6 methods)

In [120...

```
x_min, fx_val, px_val = gradient_descent_with_penalty_method(f, f_grad, p, p_grad)

([0.14426729634539417, -1.5247782207908678], Any[0.0, 0.0001, 0.00039932433010484
385, 0.0008969696570167195, 0.0015919457865737469, 0.0024832751273585417, 0.00356
9992145003199, 0.004851142807112407, 0.0063257840186619245, 0.007992983047663205
... 2.1461214801102844, 2.1156806373681887, 2.1461263085012465, 2.115684958329441
5, 2.146131150428343, 2.115689293510241, 2.146136005855604, 2.115693642873822, 2.
1461408747471094, 2.1156980063834694], Any[2.25, 2.2201, 2.190400172295816, 2.160
9008645051735, 2.1316024278822496, 2.102505216907619, 2.0736095889118173, 2.04491
59037317575, 2.0164245233992526, 1.988135811860991 ... 1.5727593806678715e-5, 4.1
6162177110622e-5, 1.572658458457504e-5, 4.1614708054475496e-5, 1.5725576317384385
e-5, 4.161320013462079e-5, 1.572456900427313e-5, 4.16116939495947e-5, 1.572356264
441134e-5, 4.161018949749692e-5])
```

wykres f od iteracji, wykres p od iteracji, szybka zbieżność

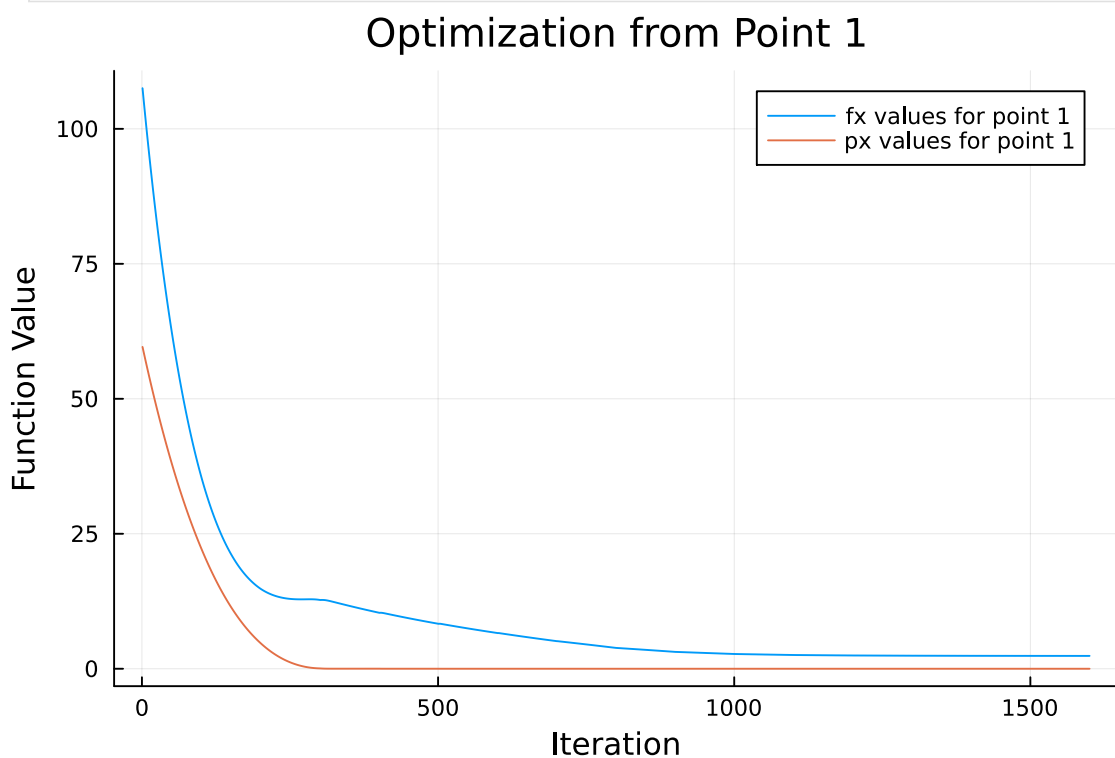
1.7 Wyniki optymalizacji z ograniczeniami dla różnych wartości parametru α

Wyświetlane są wykresy wartości funkcji f i funkcji p w zależności od kroku iteracji dla wylosowanych 10 punktów z przedziałów $x = [-5, 5]$, $y = [-5, 5]$. Wywoływana jest funkcja `gradient_descent_with_penalty_method`.

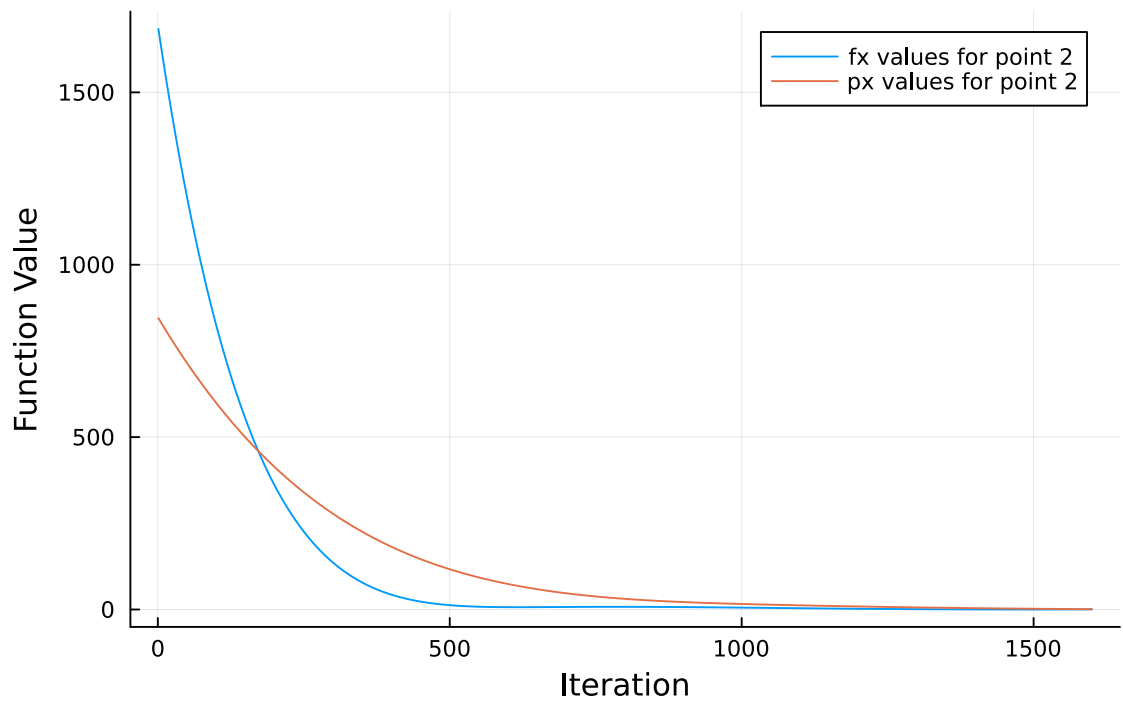
1.7.1 Szybka zbieżność - $\alpha = 0.005$

Odpowiednia wartość parametru, funkcja zbiega do minimum ograniczenia.

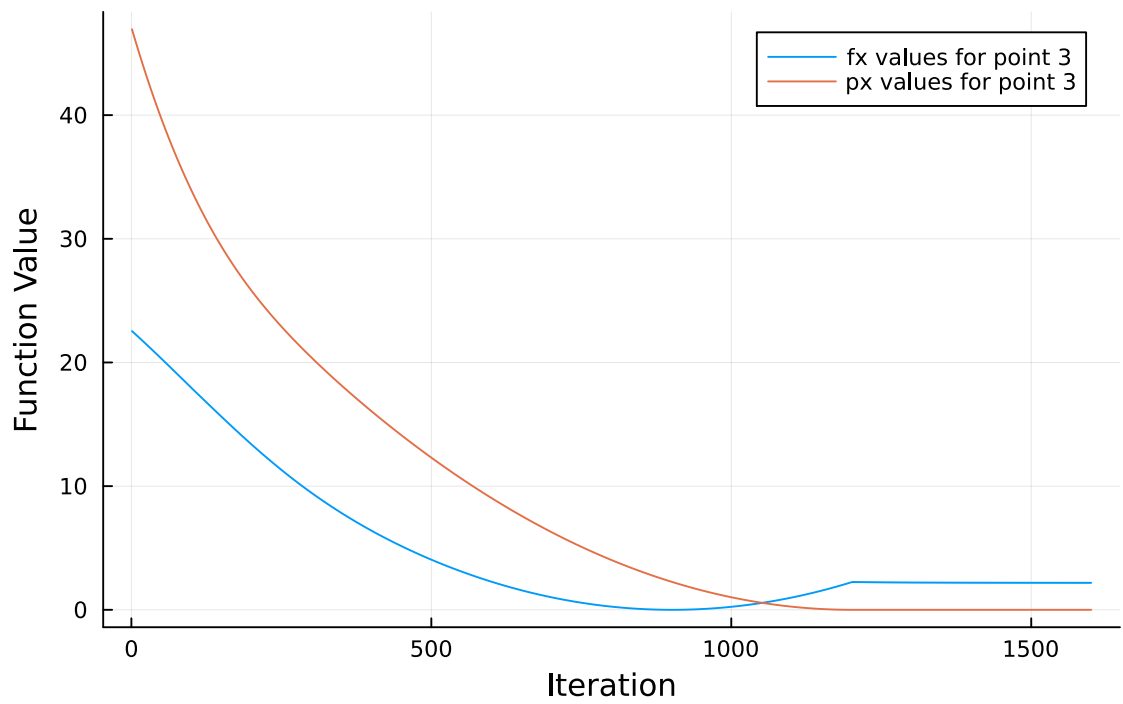
```
In [121... for i in 1:10
    x_min, fx_val, px_val = gradient_descent_with_penalty_method(f, f_grad, p, p
    plot(1:length(fx_val), fx_val, label="fx values for point $i")
    plot(1:length(px_val), px_val, label="px values for point $i")
    display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel
end
```



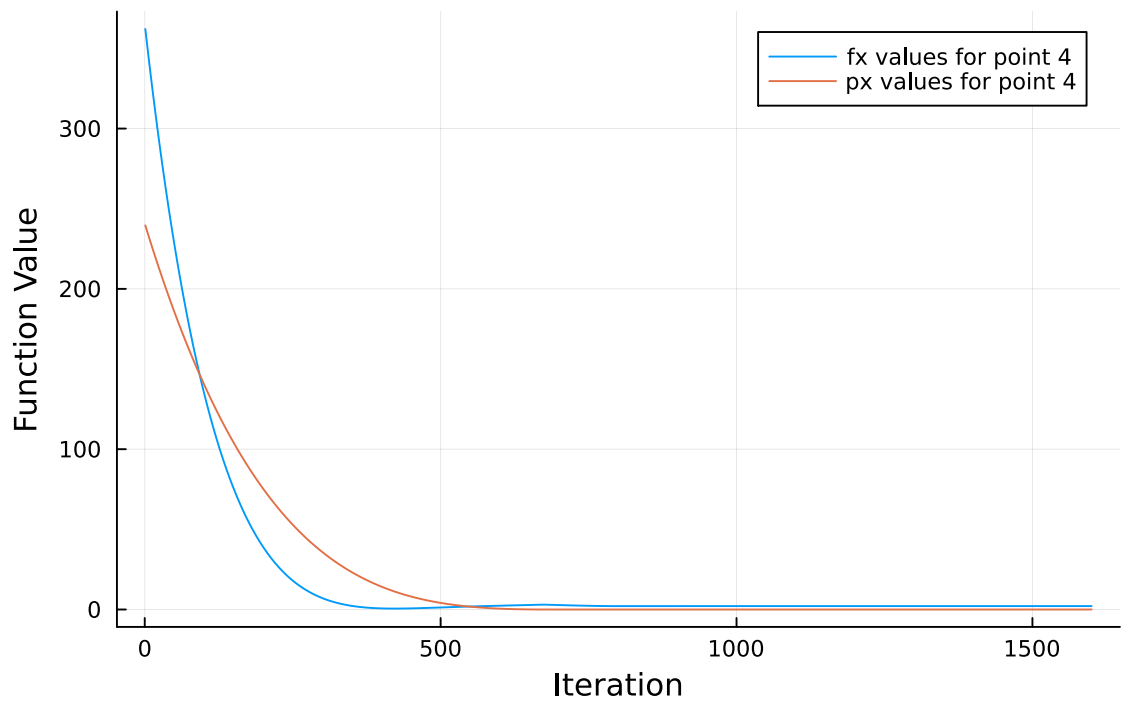
Optimization from Point 2



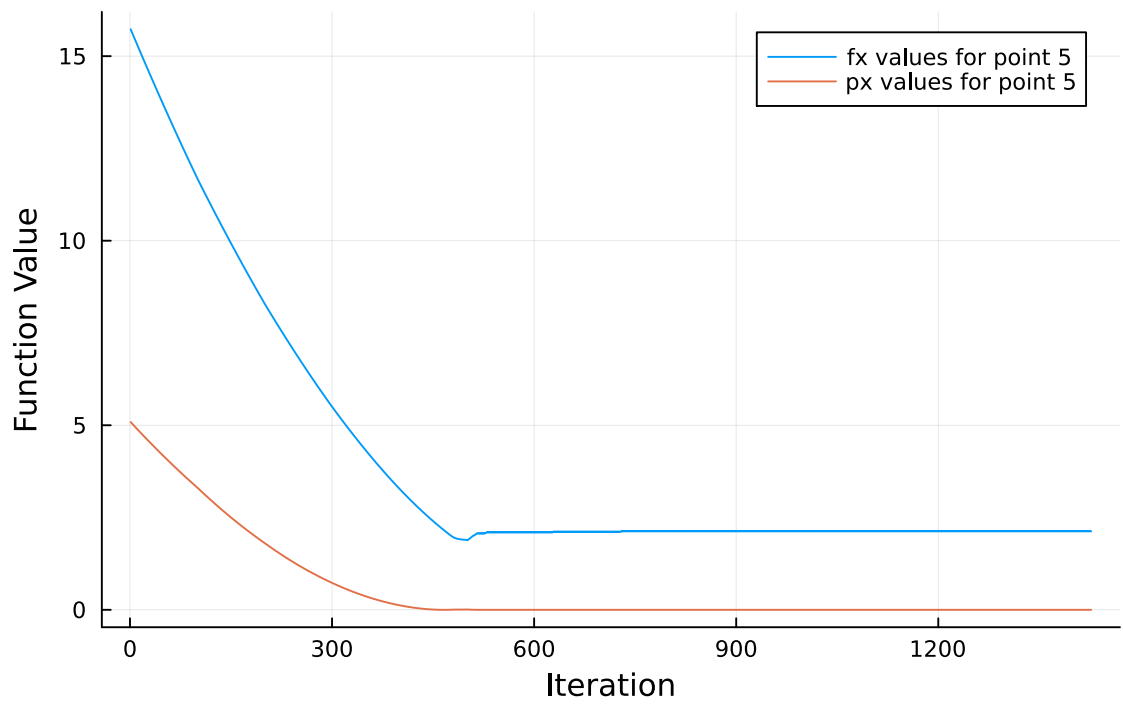
Optimization from Point 3



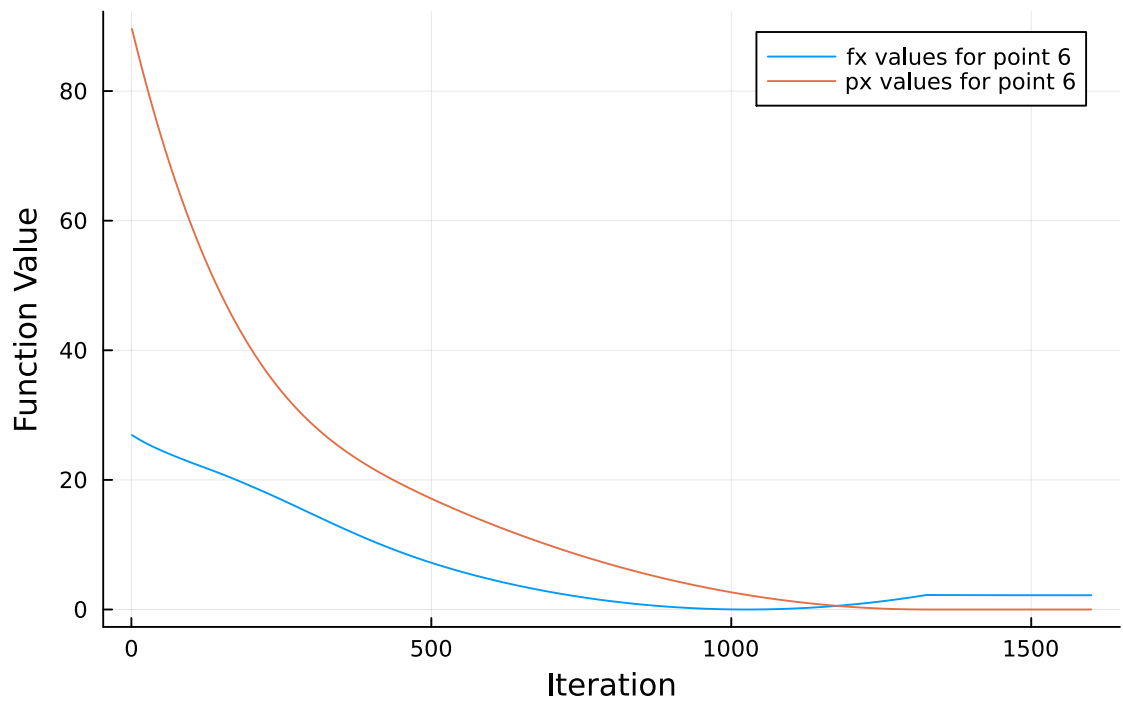
Optimization from Point 4



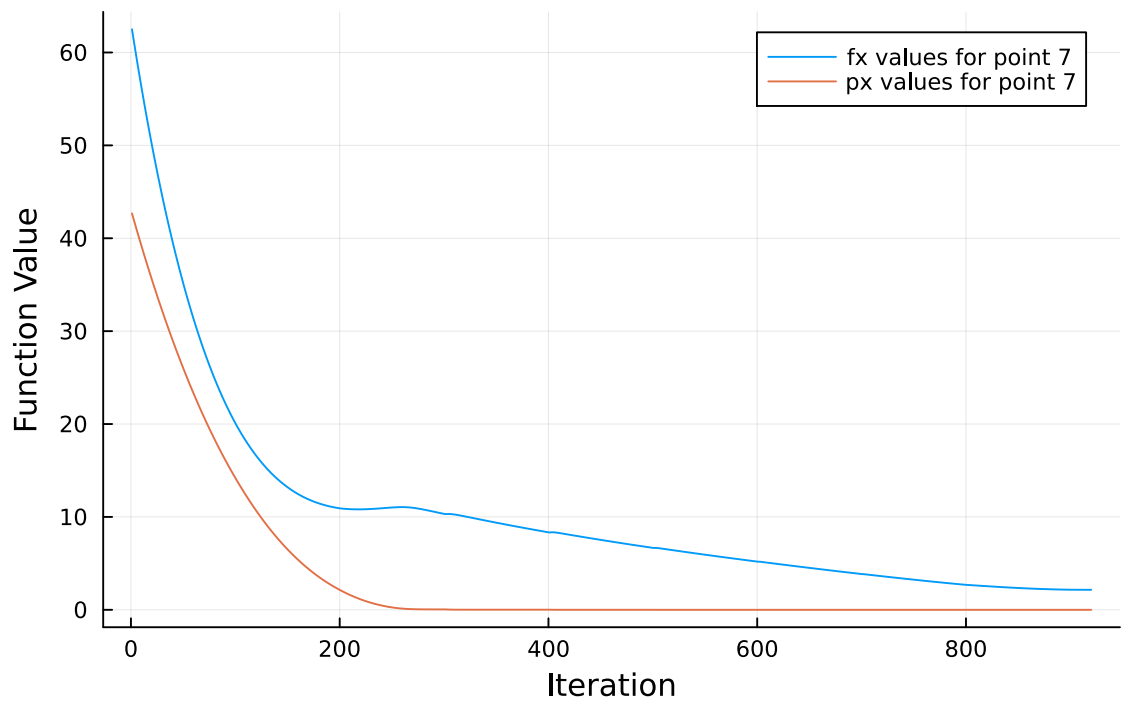
Optimization from Point 5



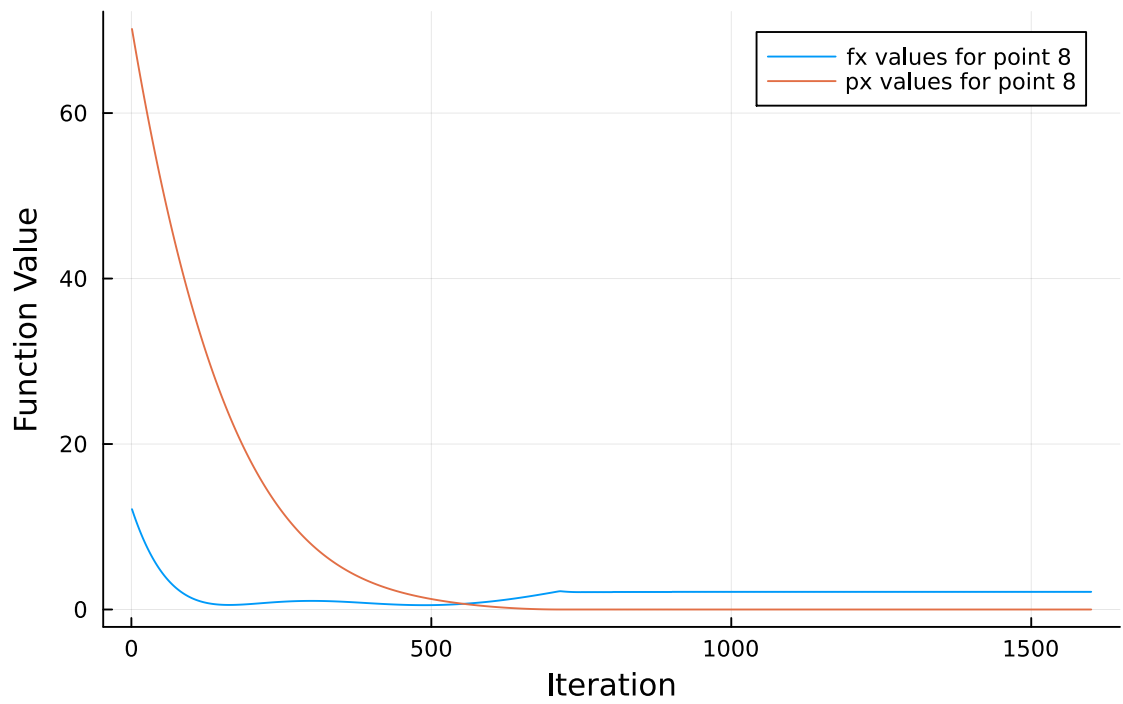
Optimization from Point 6



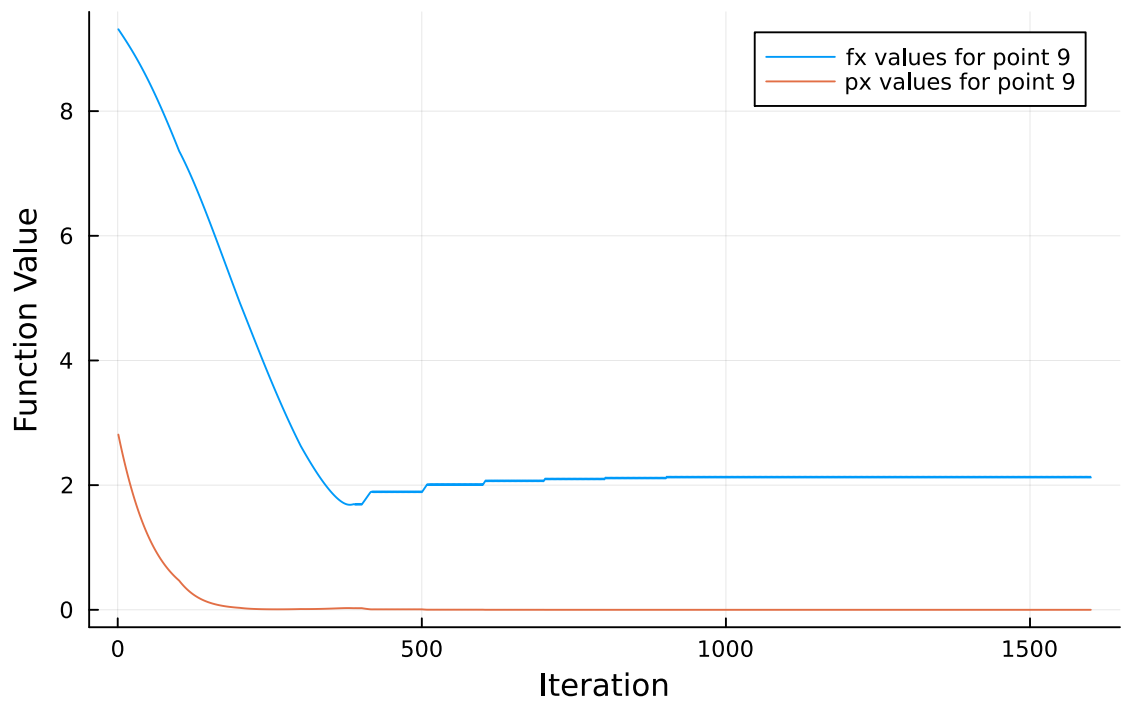
Optimization from Point 7



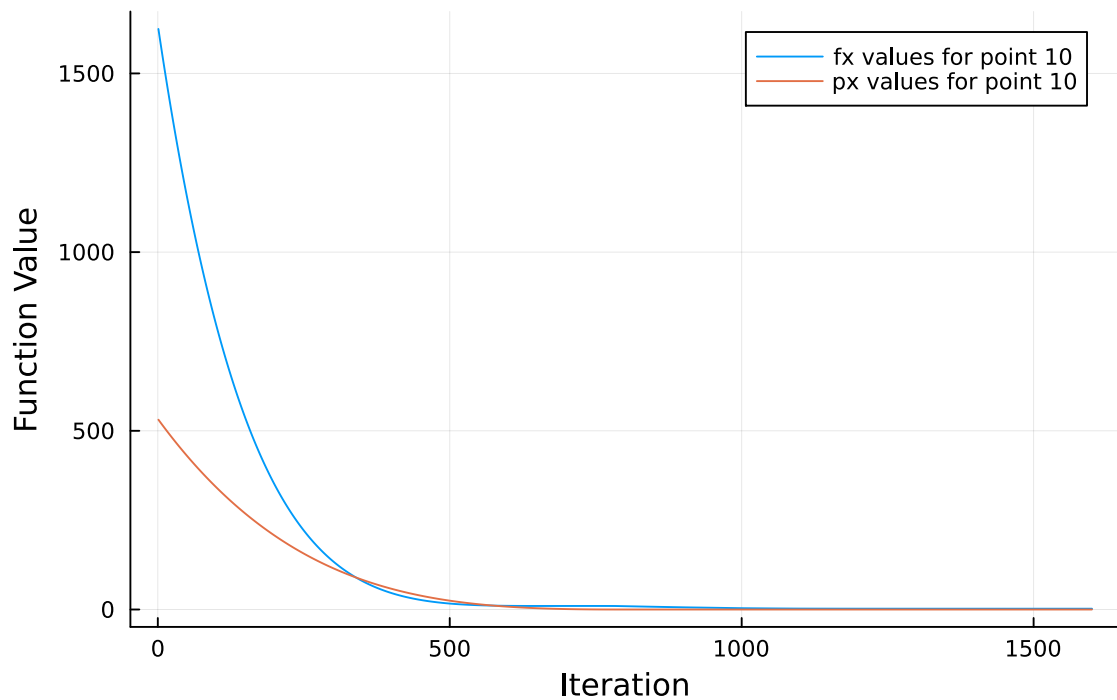
Optimization from Point 8



Optimization from Point 9



Optimization from Point 10



Wykresy 4 Można zauważyć, że w niektórych przypadkach wartości funkcji optymalizowanej f skacze do góry. Jest to spowodowane dodaniem ograniczenia, które w kolejnych iteracjach ma coraz większe znaczenie. Powiększa się parametr $\rho = 1$, ponieważ w kolejnych krokach penalty method jest mnożony przez parametr $\gamma = 2$. TO znaczy, że ograniczenie coraz bardziej wpływa na optymalizację a funkcja kary p odpowiednio maleje.

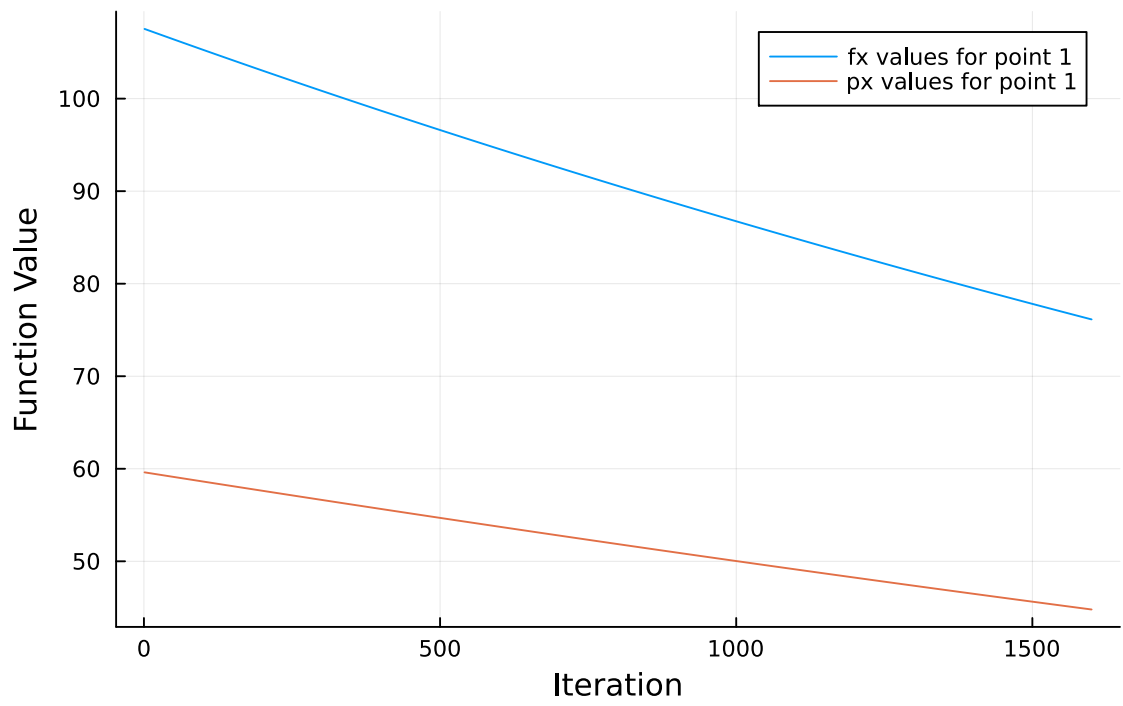
1.7.2 Powolna zbieżność - $\alpha = 1e-4$

Zbyt mała wartość parametru, funkcja nie zdąży zbiec do minimum.

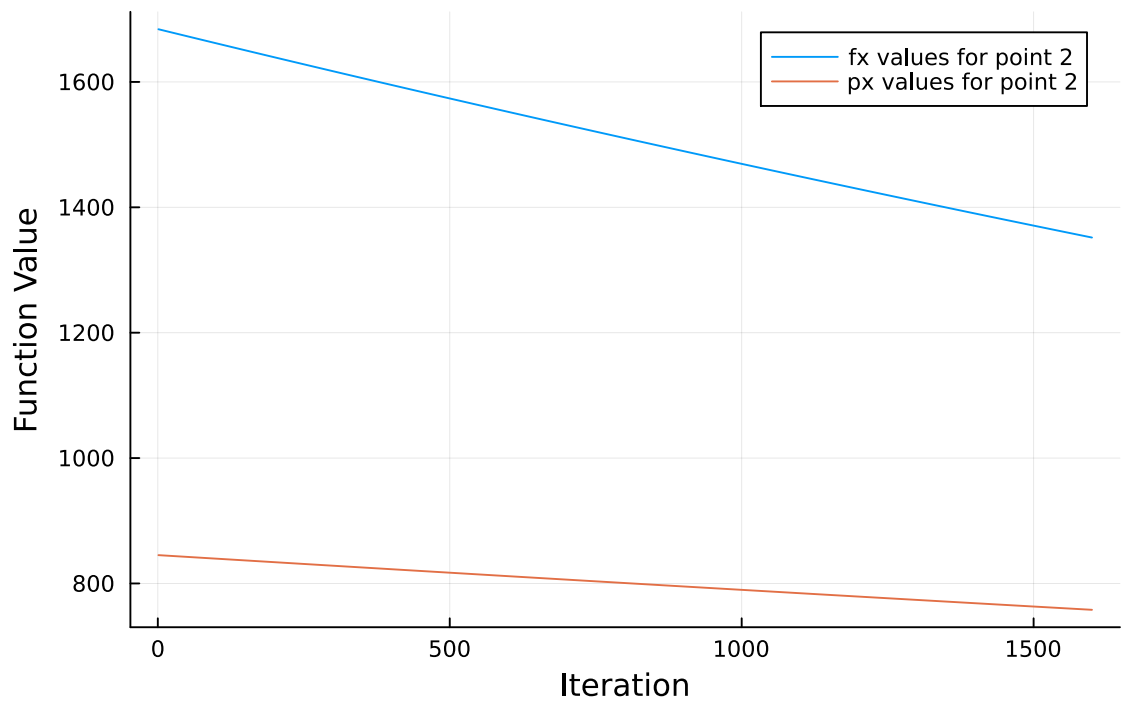
In [122...

```
for i in 1:10
    x_min, fx_val, px_val = gradient_descent_with_penalty_method(f, f_grad, p, p
    plot(1:length(fx_val), fx_val, label="fx values for point $i")
    plot(1:length(px_val), px_val, label="px values for point $i")
    display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel
end
```

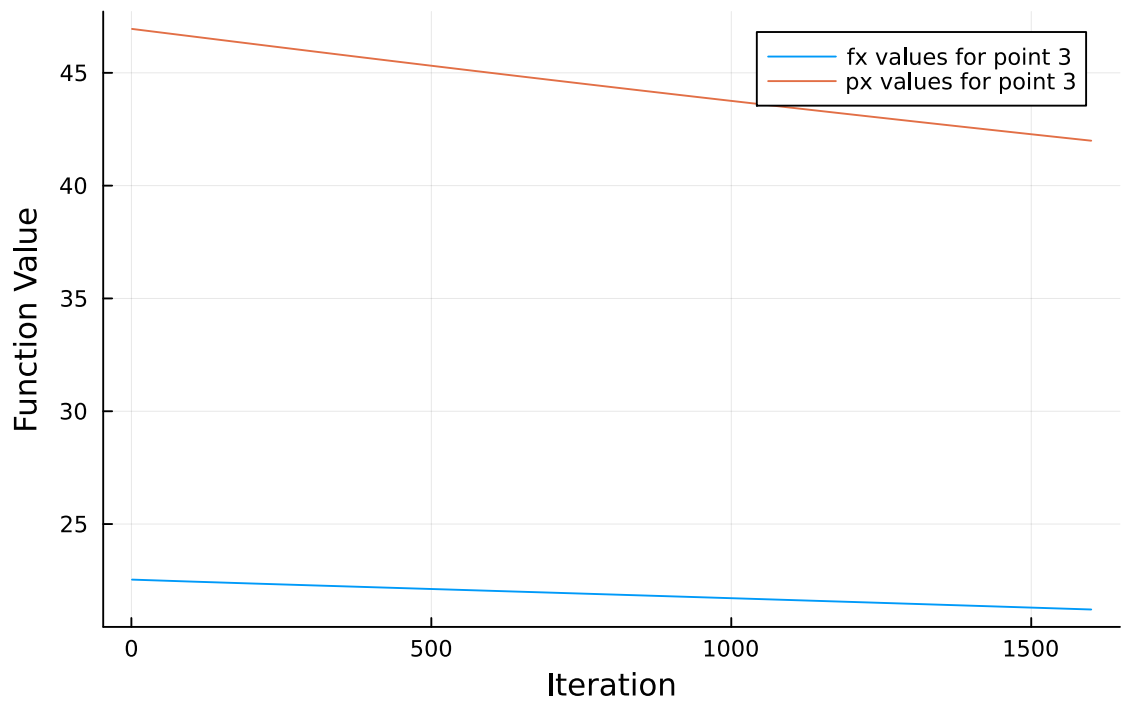
Optimization from Point 1



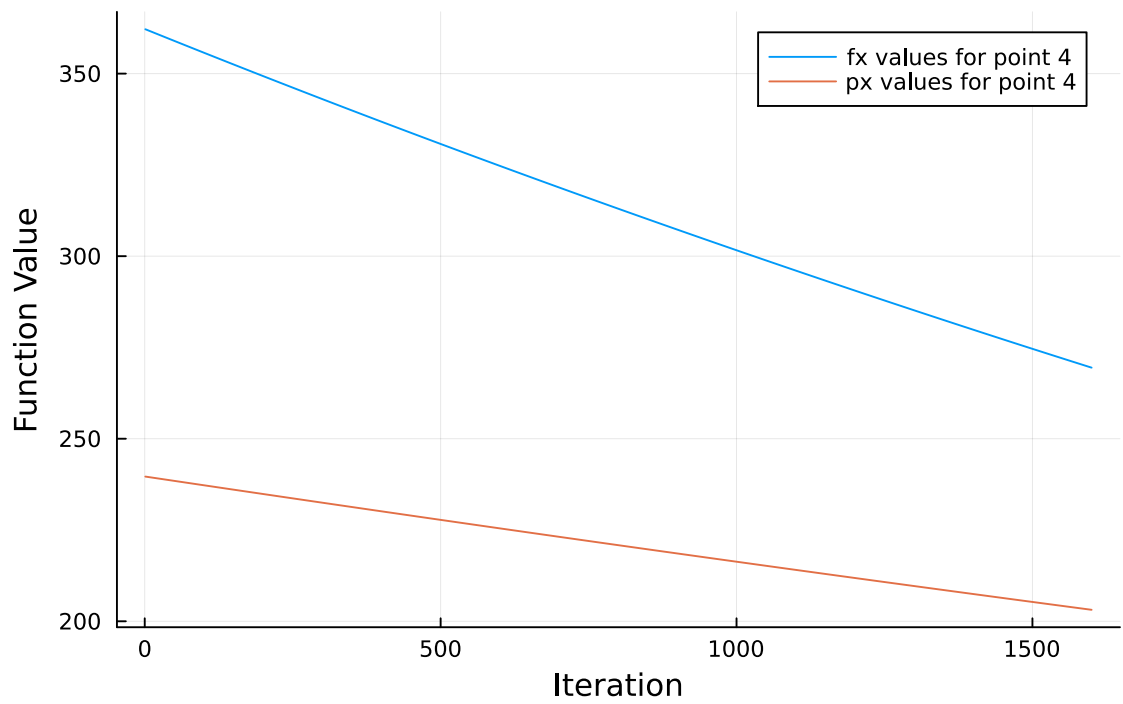
Optimization from Point 2



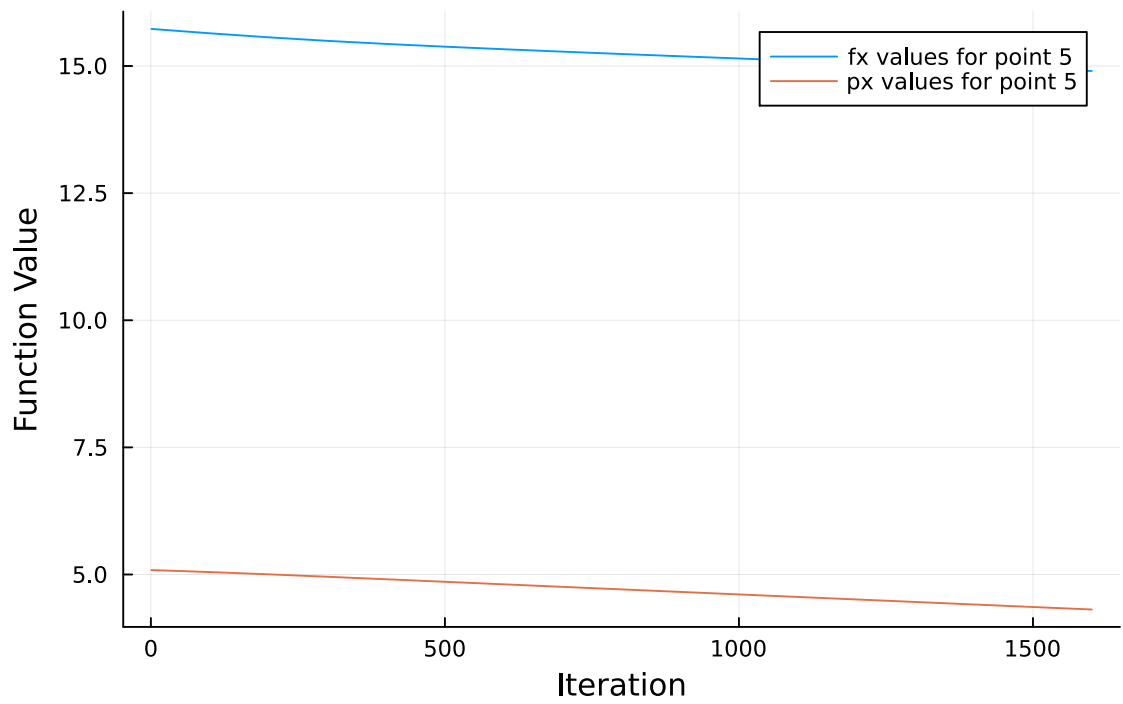
Optimization from Point 3



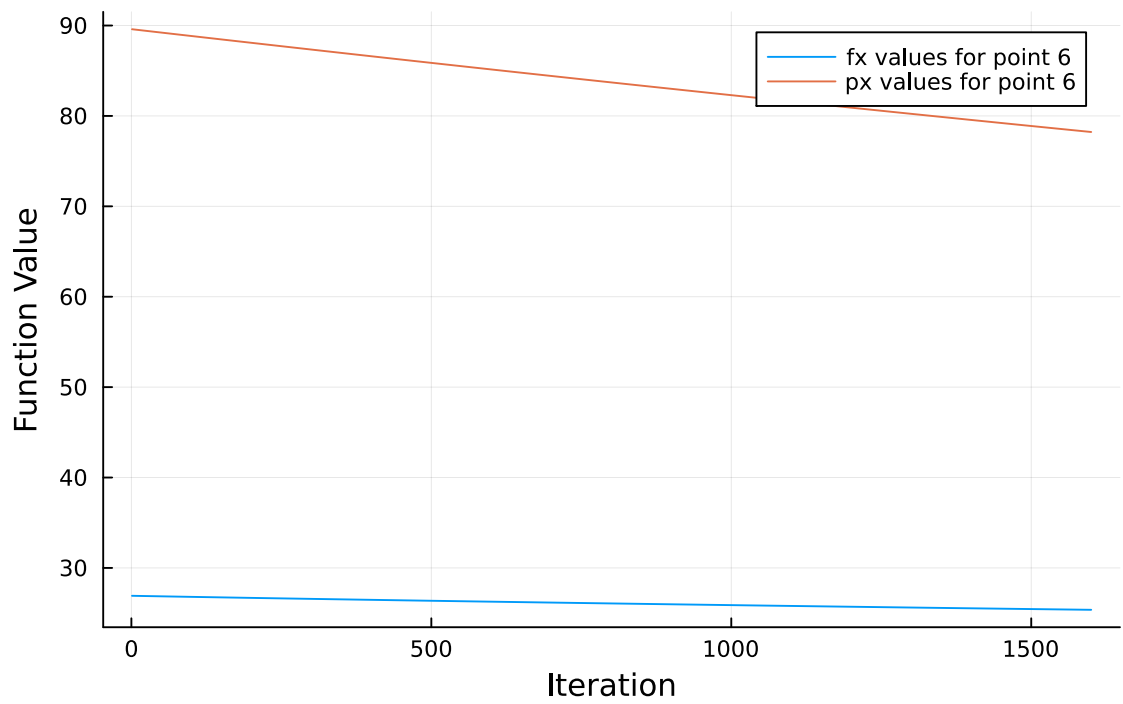
Optimization from Point 4



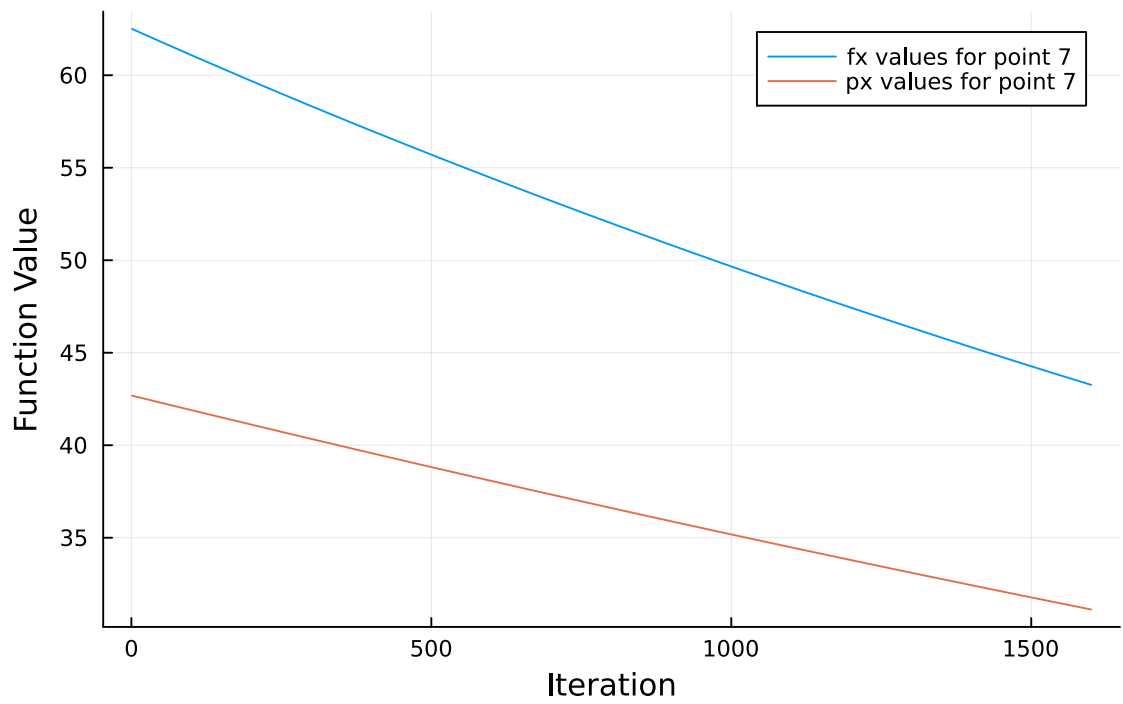
Optimization from Point 5



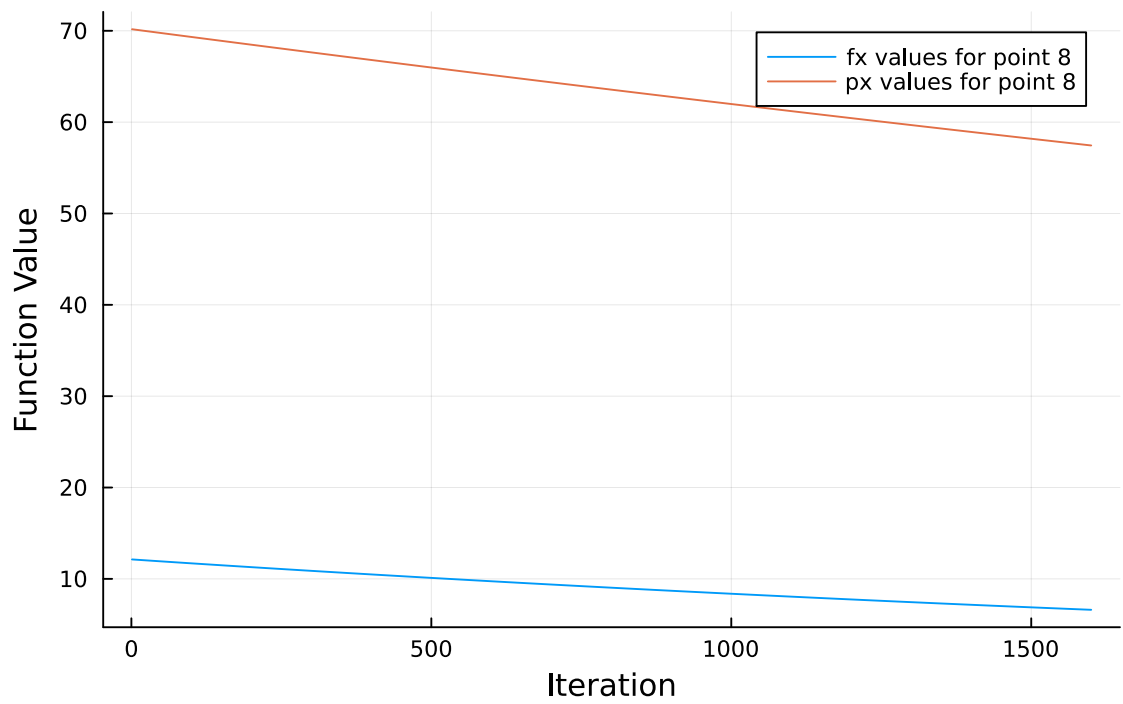
Optimization from Point 6

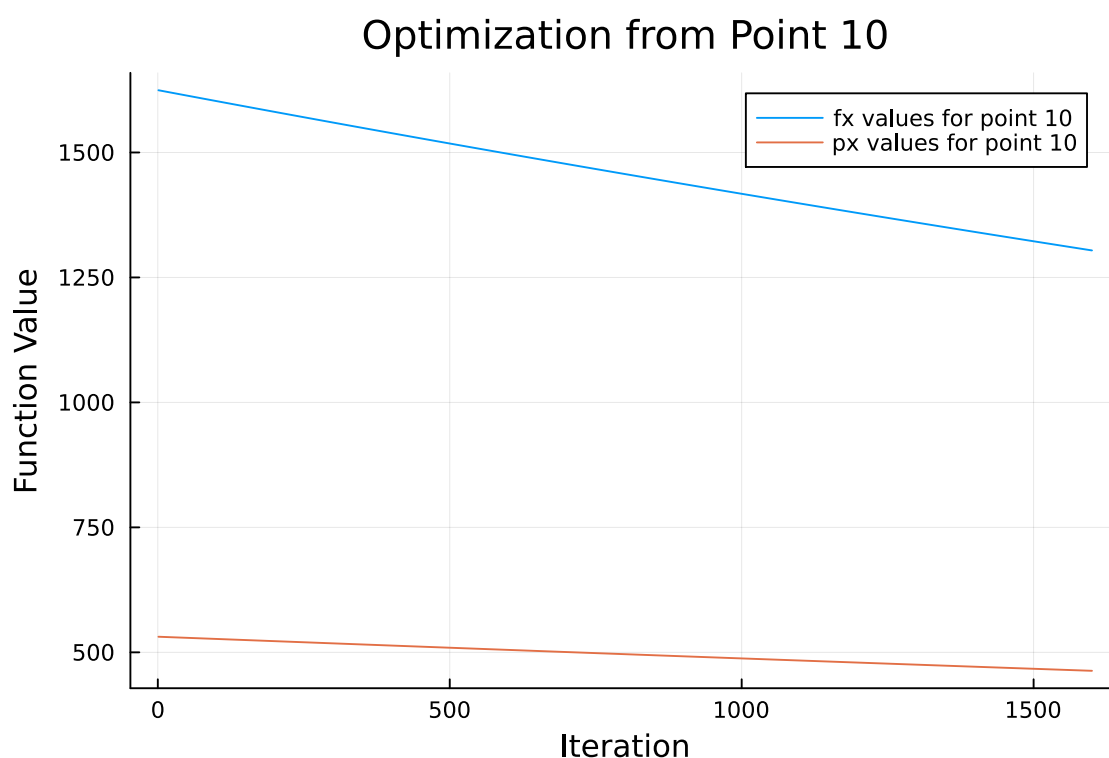
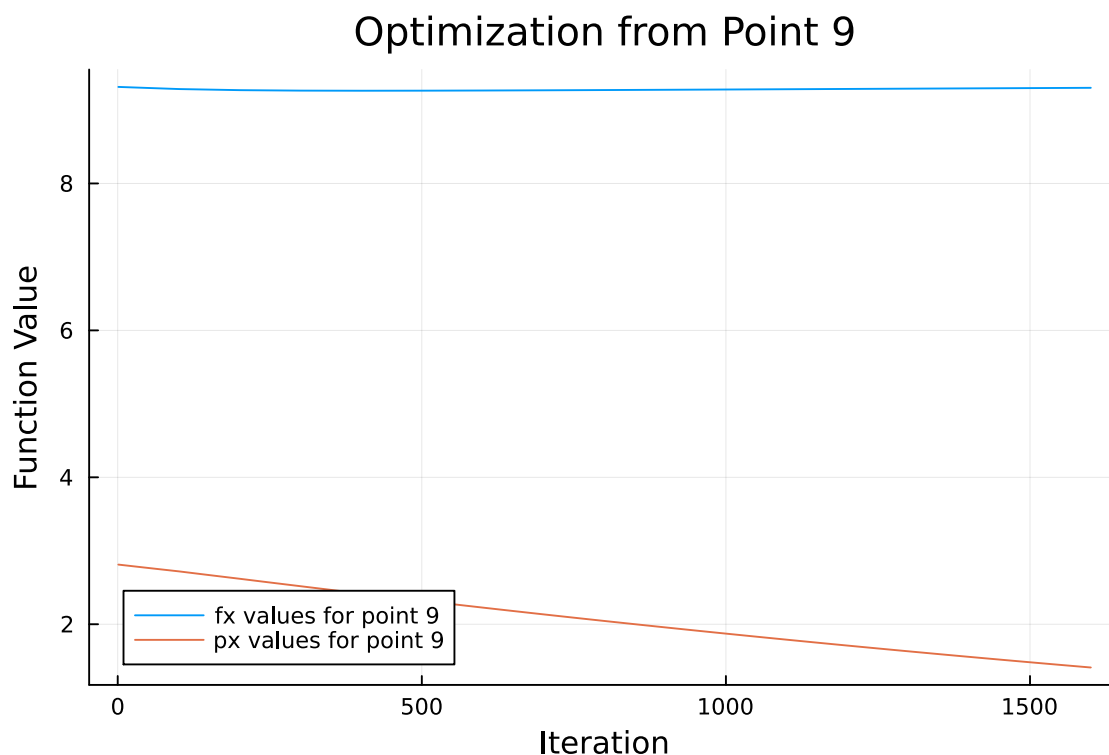


Optimization from Point 7



Optimization from Point 8





Wykresy 5 - zarówno funkcja p jak i funkcja f bardzo wolno maleją

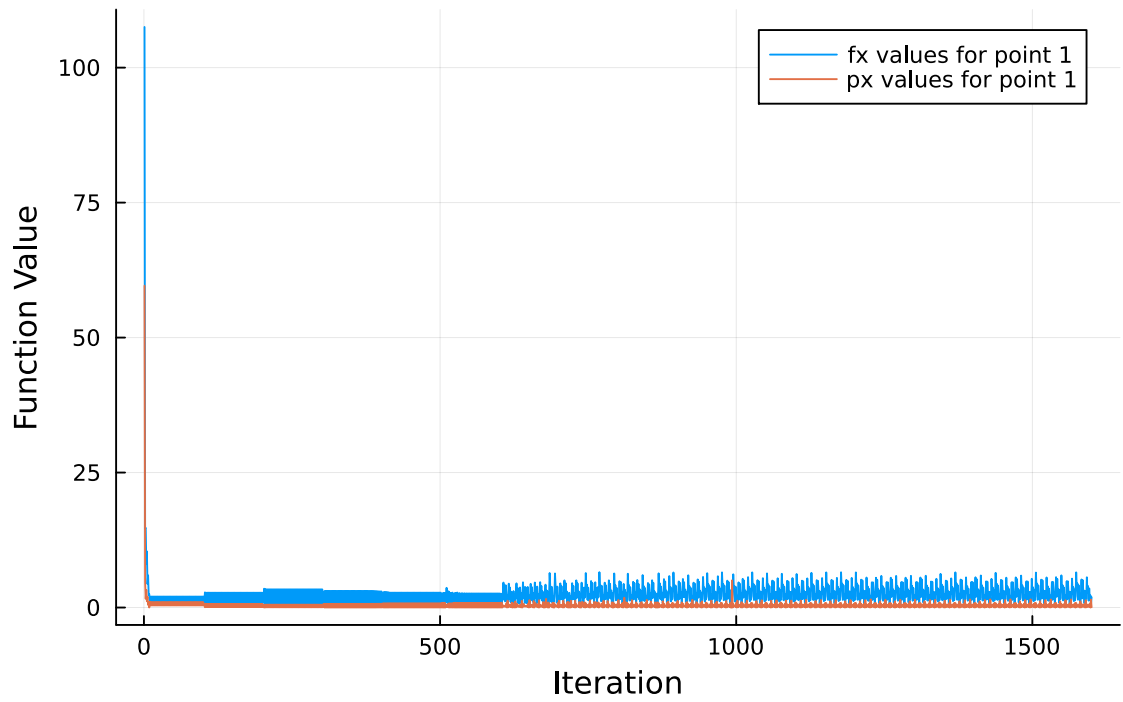
1.7.3 Brak zbieżności - $\alpha = 1$

Zbyt duża wartość parametru, funkcja oddala się od minimum

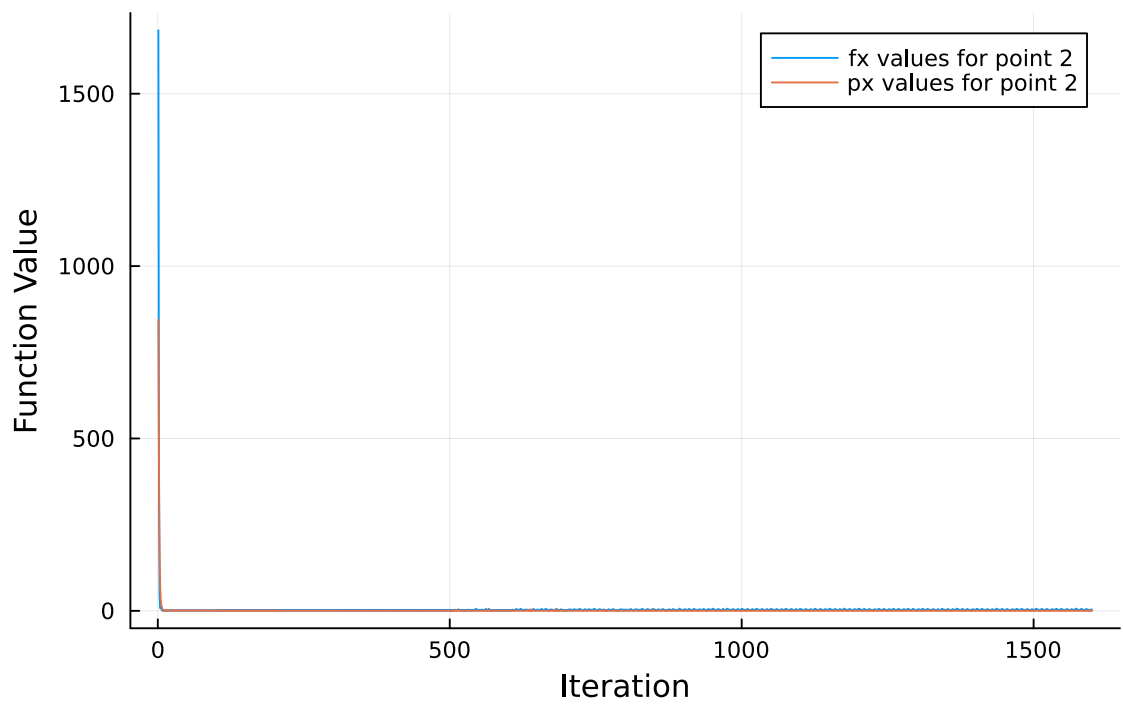
```
In [123... for i in 1:10
    x_min, fx_val, px_val = gradient_descent_with_penalty_method(f, f_grad, p, p
    plot(1:length(fx_val), fx_val, label="fx values for point $i")
    plot(1:length(px_val), px_val, label="px values for point $i")
```

```
display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel=
end
```

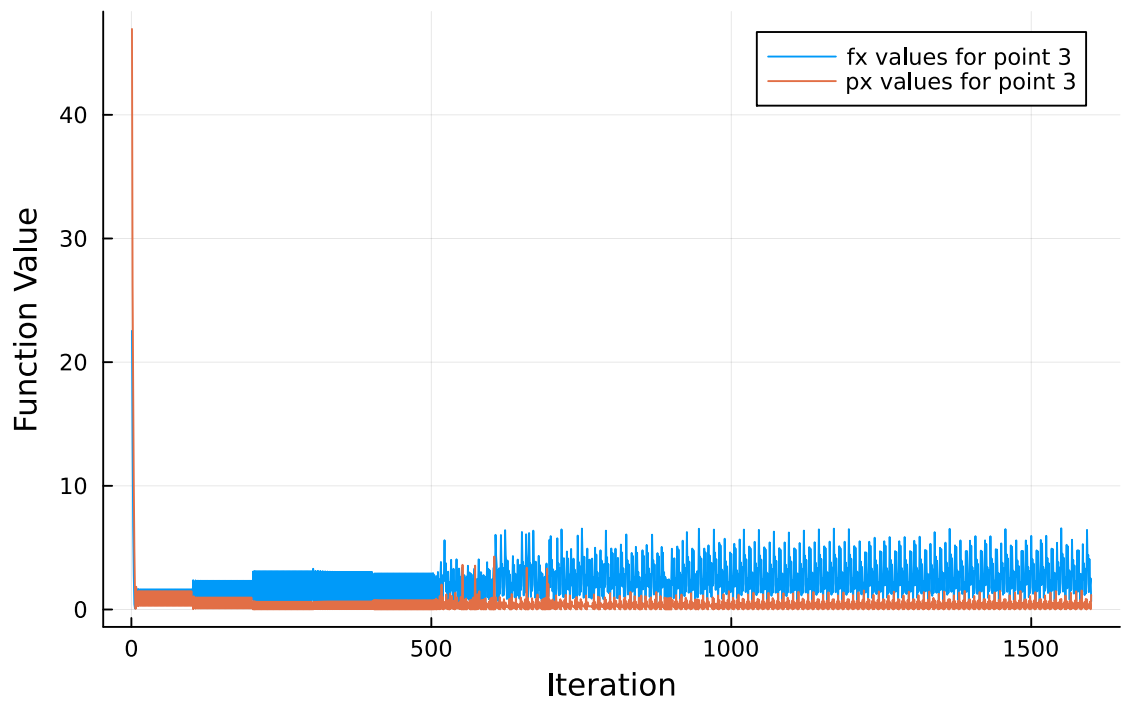
Optimization from Point 1



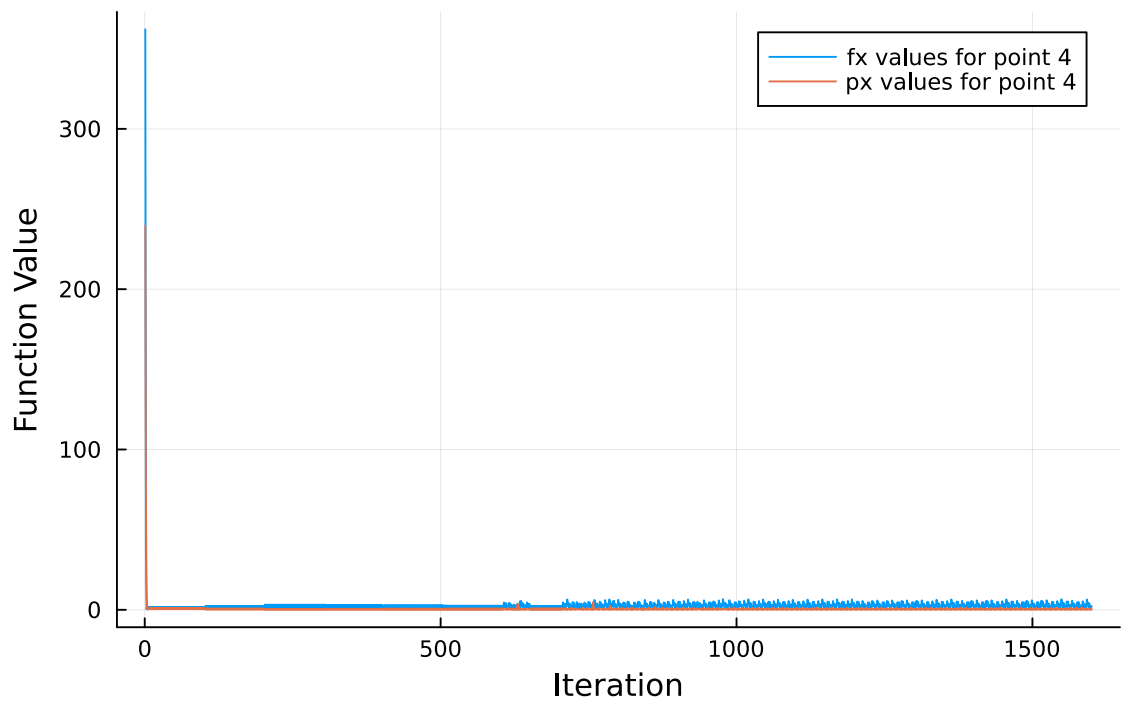
Optimization from Point 2



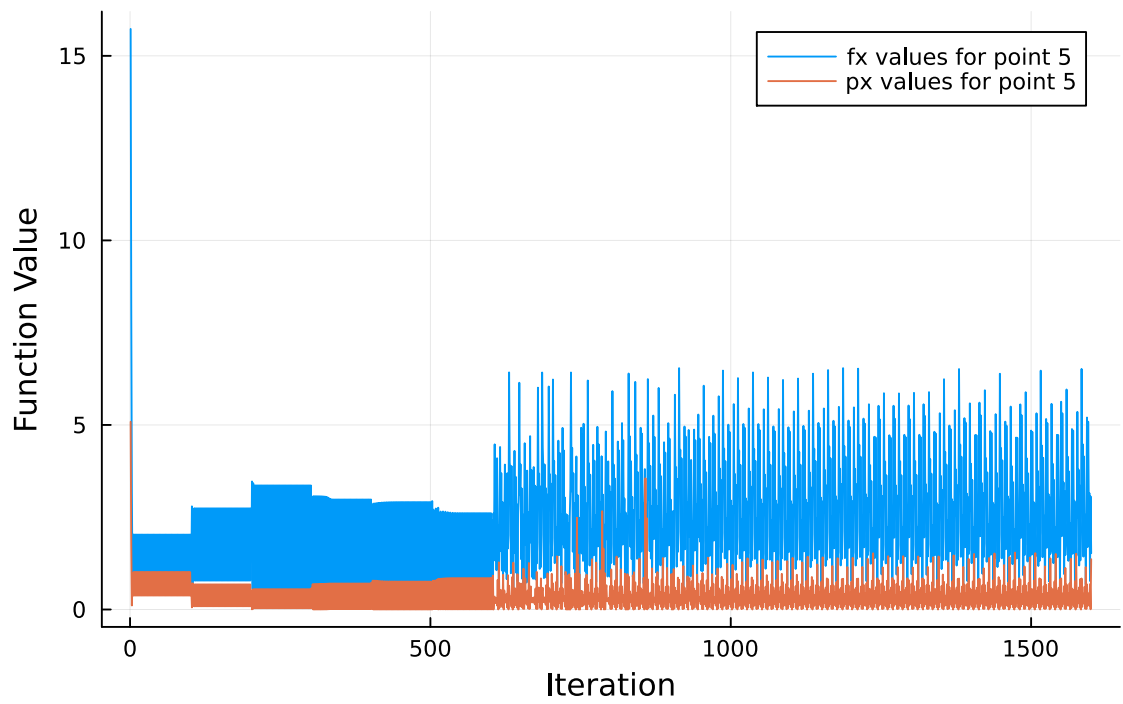
Optimization from Point 3



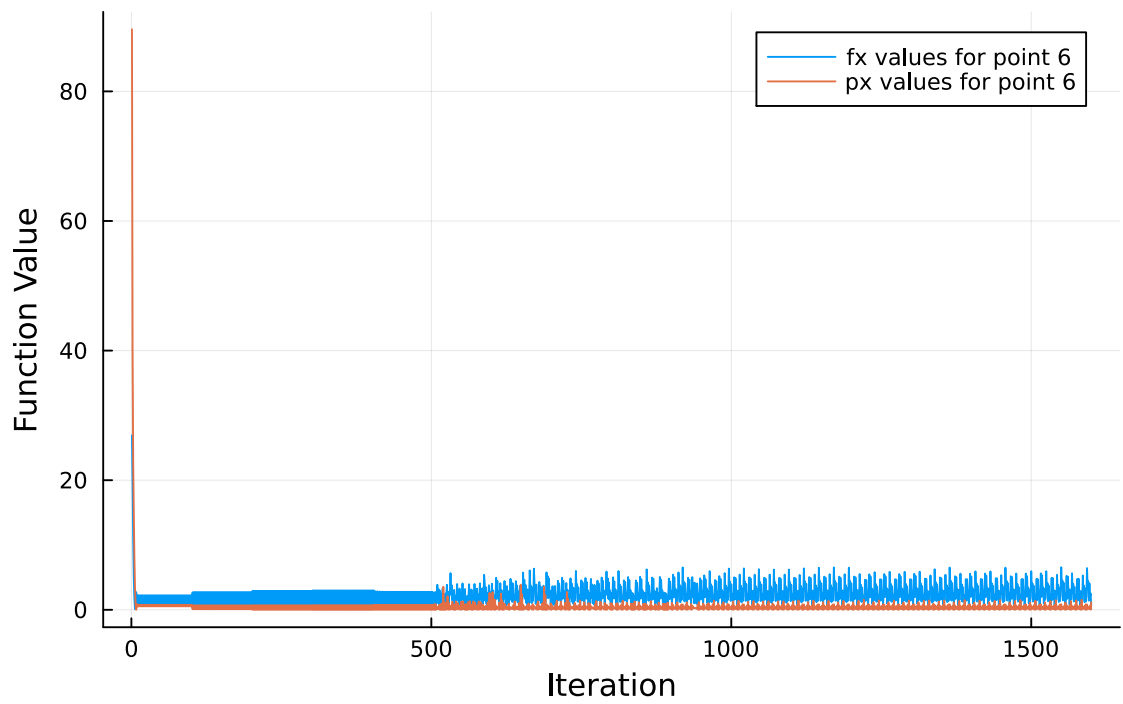
Optimization from Point 4



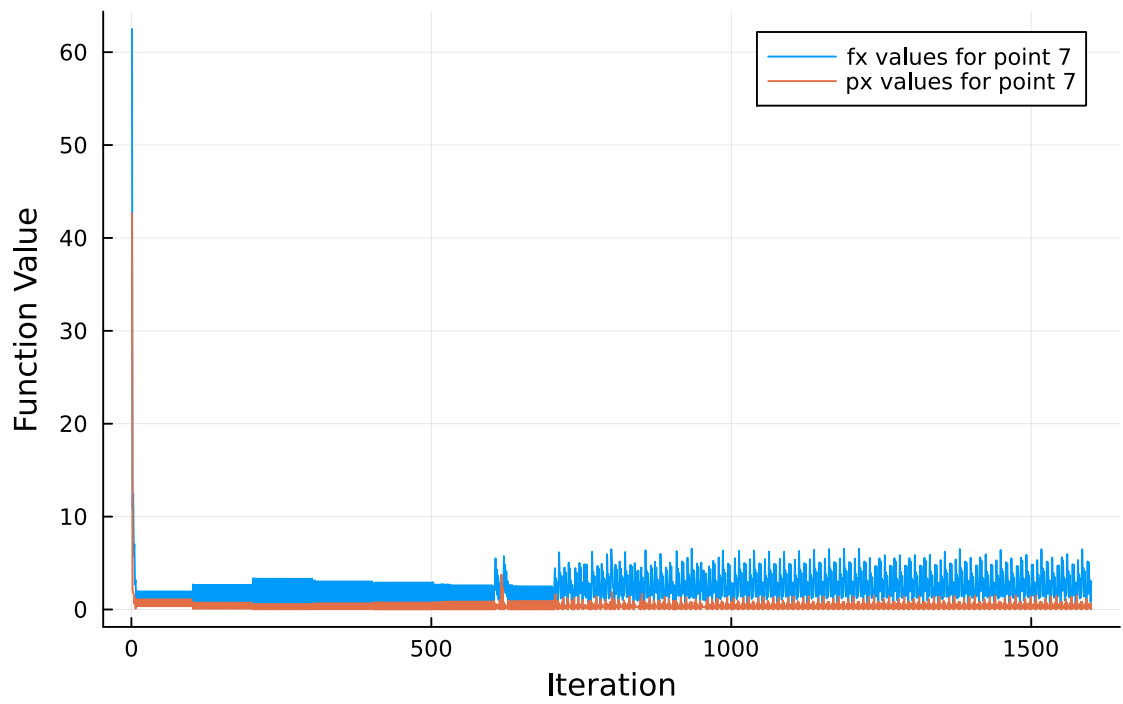
Optimization from Point 5



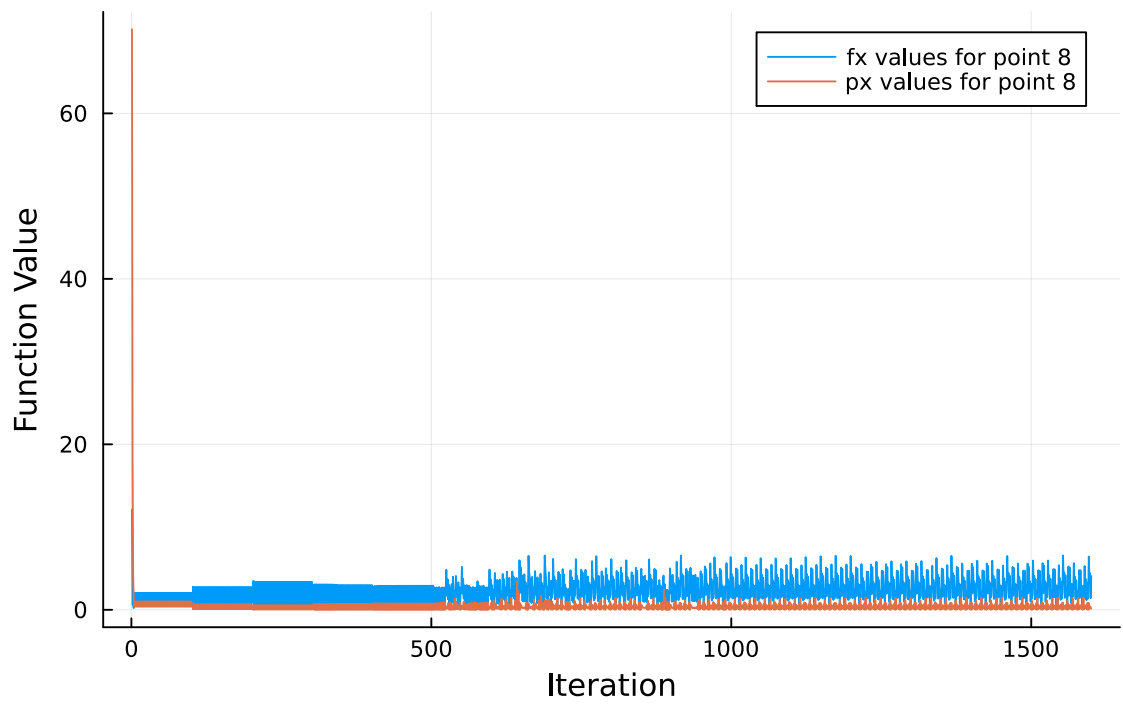
Optimization from Point 6



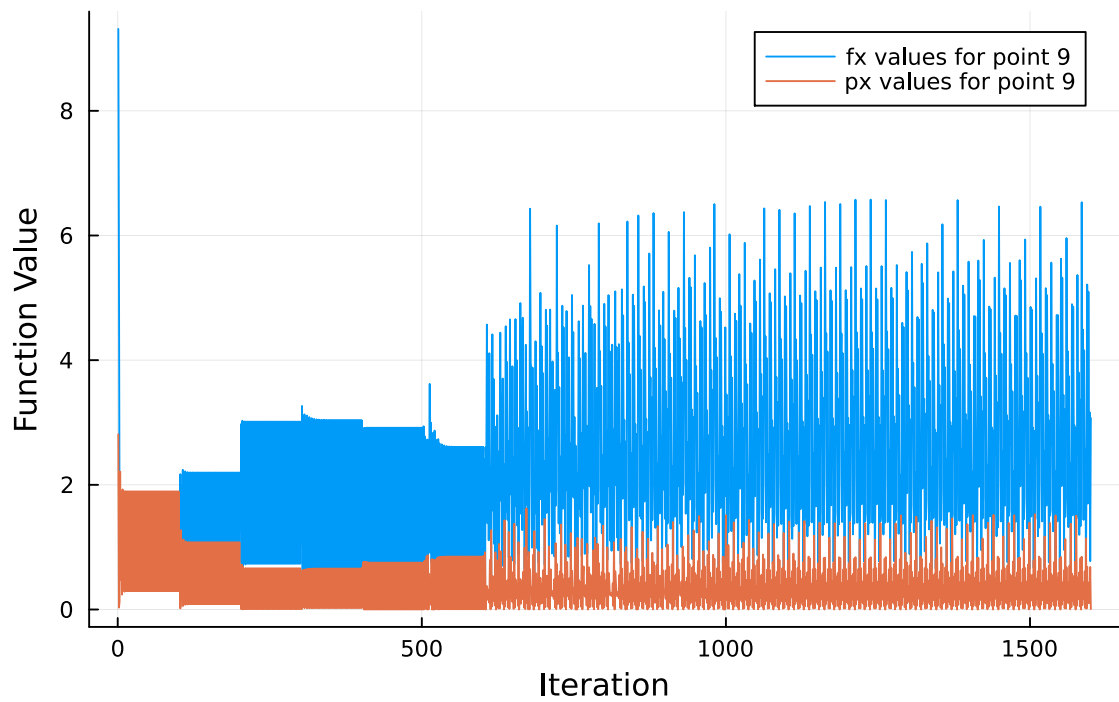
Optimization from Point 7



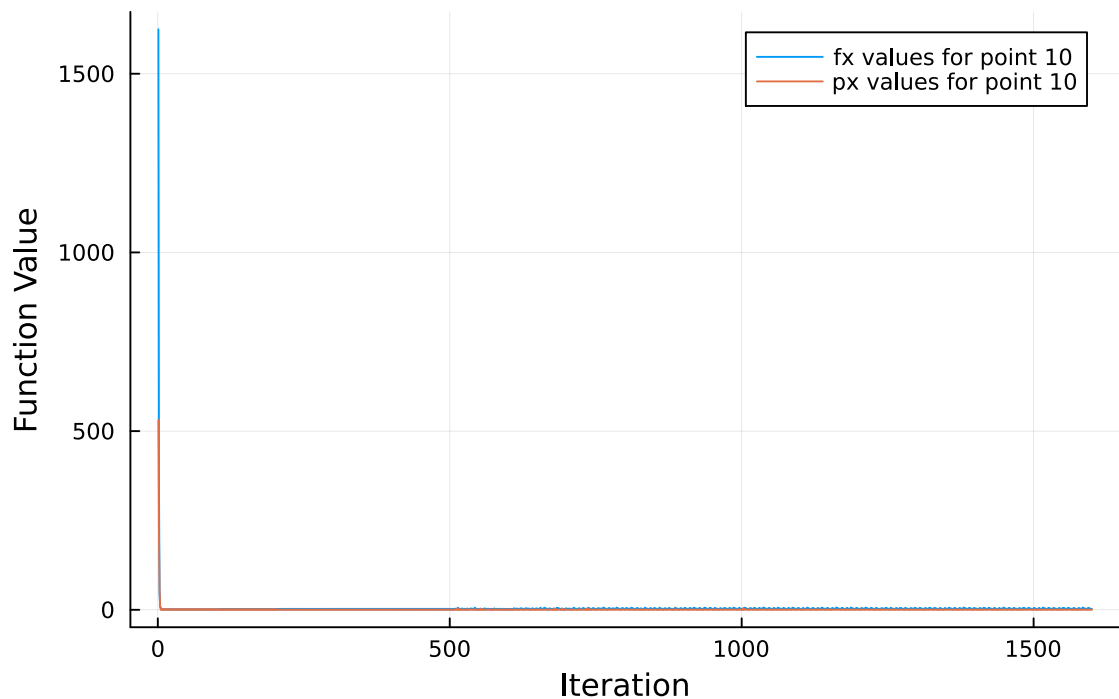
Optimization from Point 8



Optimization from Point 9



Optimization from Point 10



Wykresy 6 Widoczne skoki zarówno dla funkcji f , jak i funkcji p , które nie prowadzą do optymalizacji.

1.8 Conjugate gradient descent with penalty method

```
In [124... function armijo_line_search(f, ∇f, x, d; α0=1.0, c=1e-4, τ=0.5)
    α = α0
    fx = f(x)
    gx = ∇f(x)
```

```

slope = dot(gx, d)

# Compute  $x + \alpha * d$  inline (this allocates a new array each time)
while f(x +  $\alpha * d$ ) > fx + c *  $\alpha * slope$ 
     $\alpha *= \tau$ 
    if  $\alpha < 1e-14$ 
        return 1e-14
    end
end
return  $\alpha$ 
end

# De-optimized penalty-based nonlinear conjugate gradient method.
function penalty_ncg_method(
    f, f_grad,
    p, p_grad,
    fp, fp_grad,
    x0;
    k_max=10,
     $\rho=1.0$ ,
     $\gamma=2.0$ ,
    atol=1e-8,
    max_iter=50,
    tol=1e-6
)
    # Use a non-optimized copy: each update creates a new vector.
    x = copy(x0)

    # Use dynamically growing arrays (no preallocation)
    f_values = []
    p_values = []

    for k in 1:k_max
        r = -fp_grad(x,  $\rho$ )
        d = copy(r)

        for i in 1:max_iter
            nr = norm(r)
            if nr < tol
                break
            end

             $\alpha$  = armijo_line_search(
                (xx) -> fp(xx,  $\rho$ ),
                (xx) -> fp_grad(xx,  $\rho$ ),
                x, d;
                 $\alpha_0 = 0.3$ ,
                c = 1e-4,
                 $\tau = 0.5$ 
            )

            # Instead of in-place update, create a new array each time.
            x = x +  $\alpha * d$ 

            push!(f_values, f(x))
            push!(p_values, p(x))

            r_new = -fp_grad(x,  $\rho$ )
             $\beta = \text{norm}(r\_new)^2 / nr^2$ 

```



```

        if (i % 10) == 0
            d = copy(r_new)
        else
            # Compute new direction without in-place update.
            d = r_new +  $\beta$  * d
        end

        r = r_new
    end

    # Check if constraint is satisfied.
    if p(x) < atol
        return x, f_values, p_values
    end

     $\rho$  *=  $\gamma$ 
end

return x, f_values, p_values
end

```

penalty_ncg_method (generic function with 2 methods)

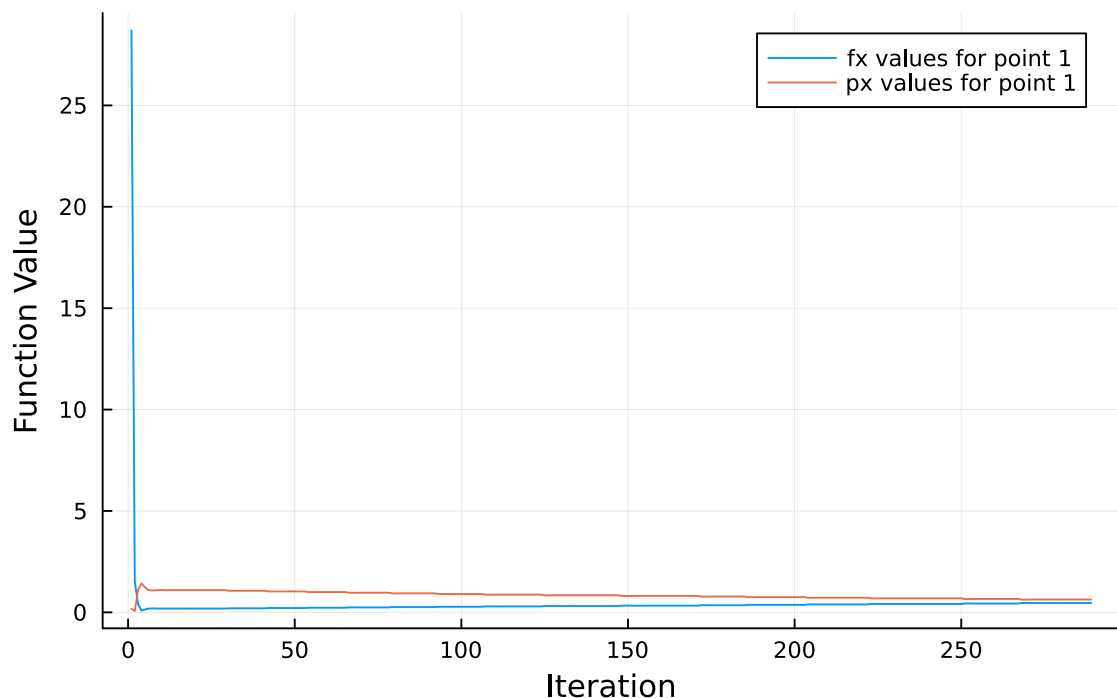
In [140...

```

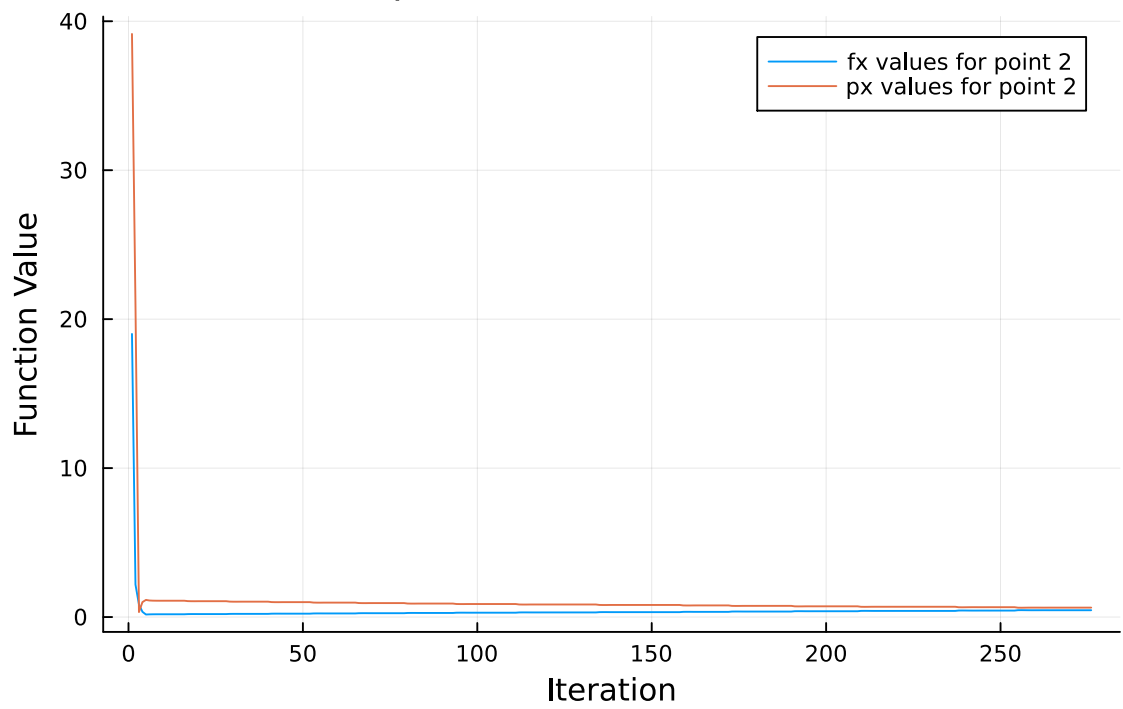
for i in 1:10
    x0 = points[i]
    x_opt, fx_vals, px_vals = penalty_ncg_method(f, f_grad, p, p_grad, fp, fp_grad)
    plot(1:length(fx_vals), fx_vals, label="fx values for point $i")
    plot(1:length(px_vals), px_vals, label="px values for point $i")
    display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel="Function Value"))
end

```

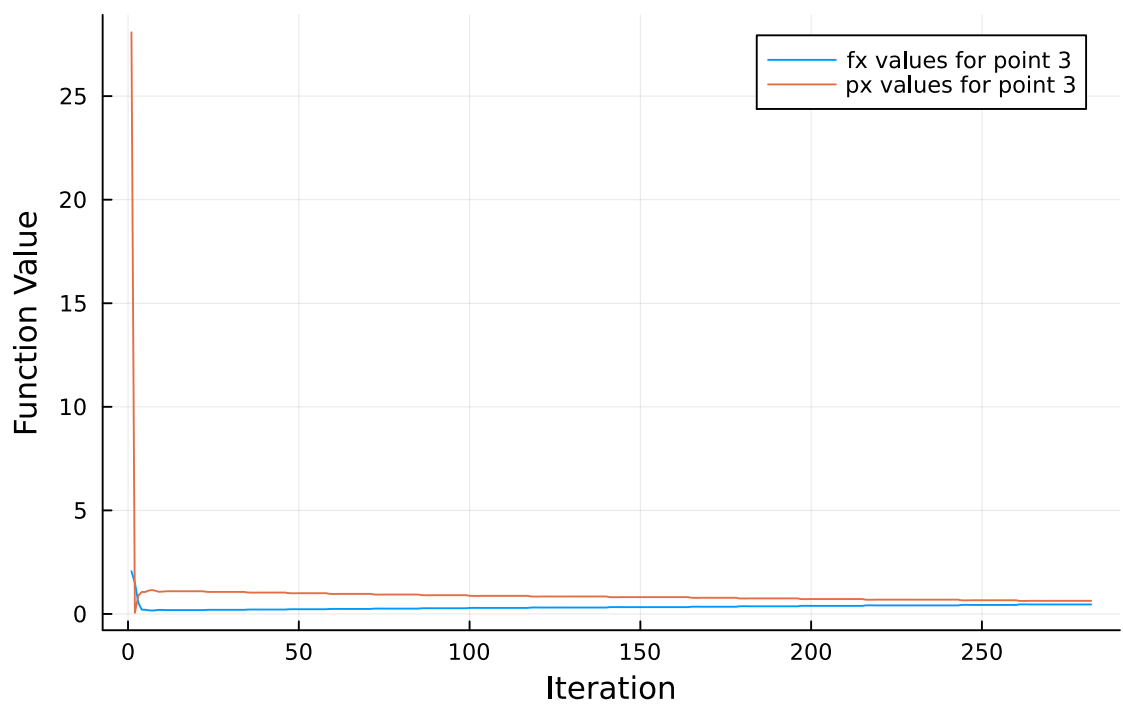
Optimization from Point 1



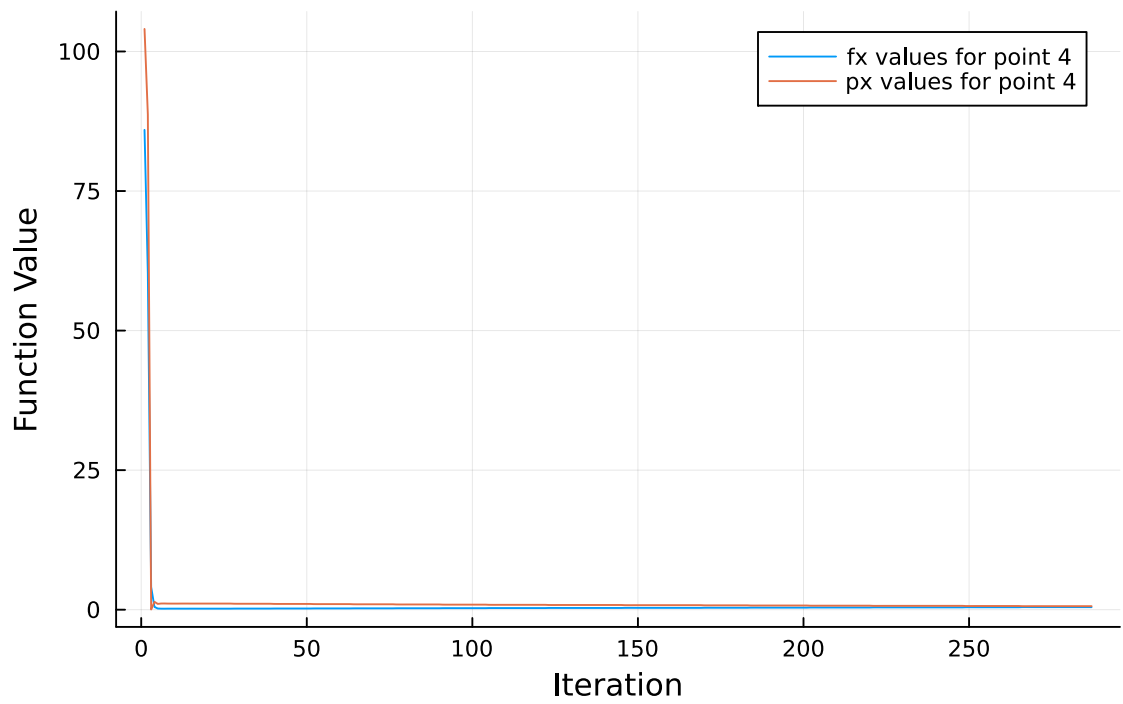
Optimization from Point 2



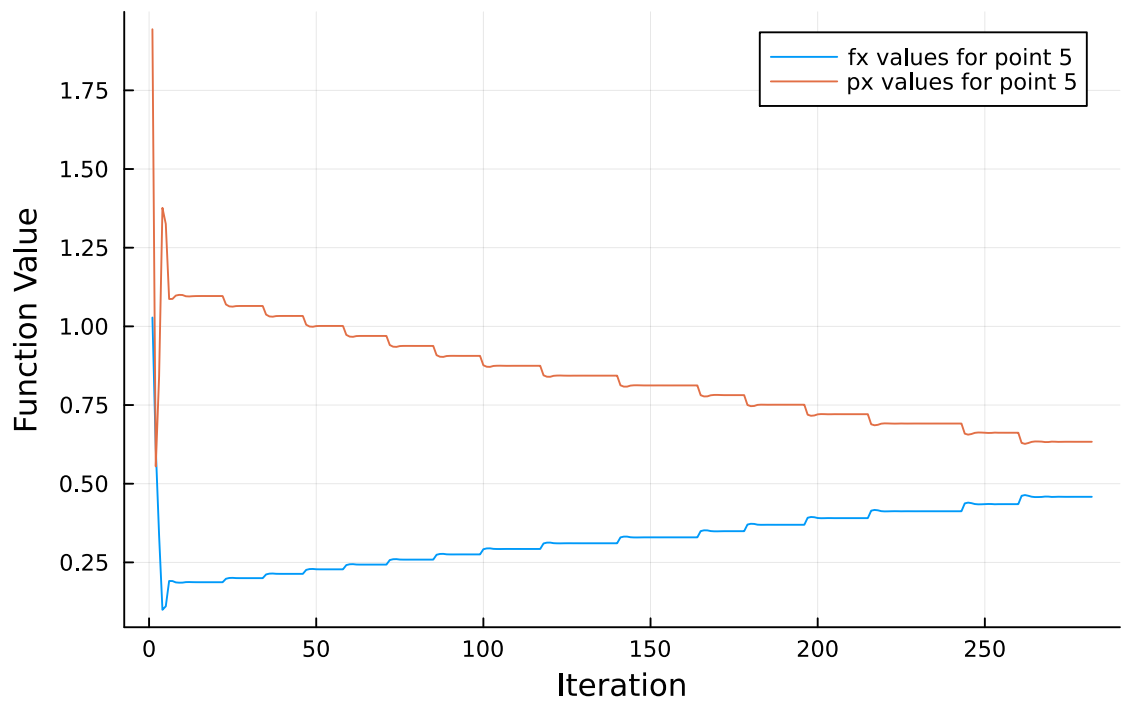
Optimization from Point 3



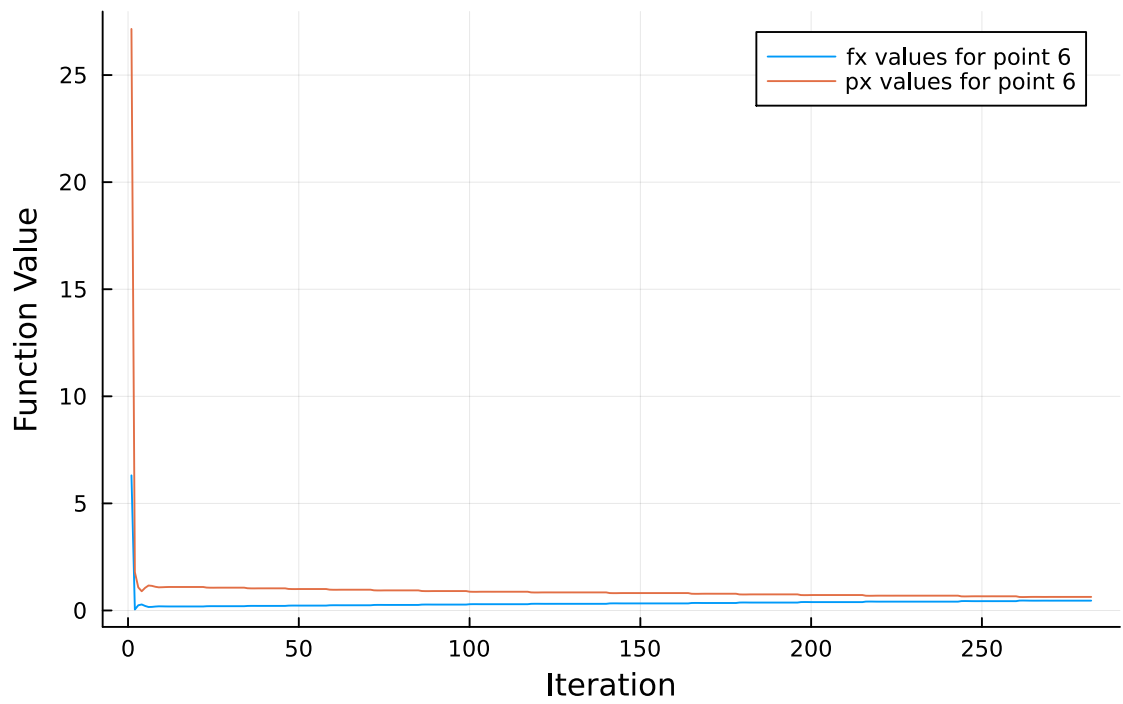
Optimization from Point 4



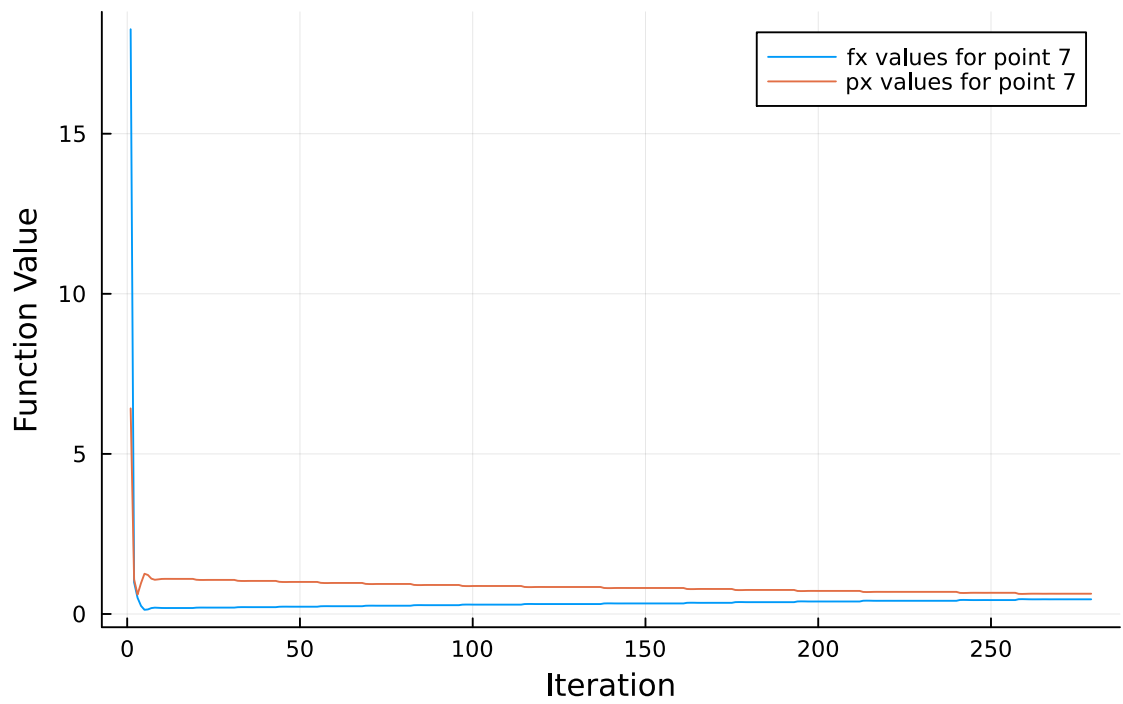
Optimization from Point 5



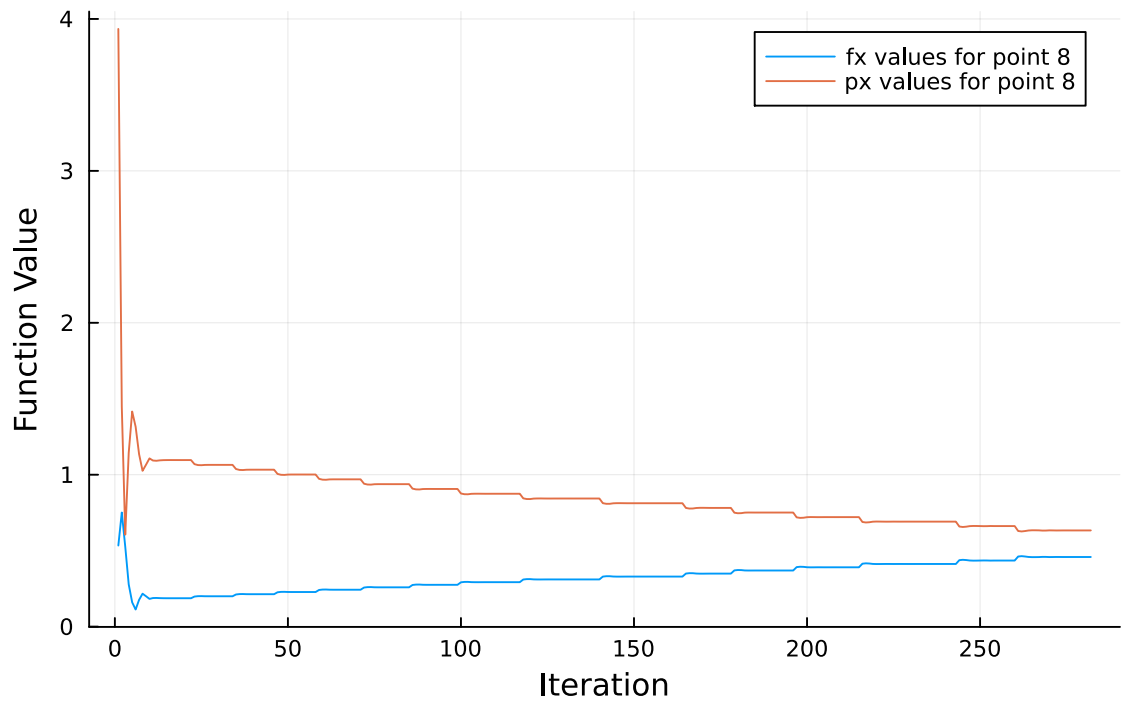
Optimization from Point 6



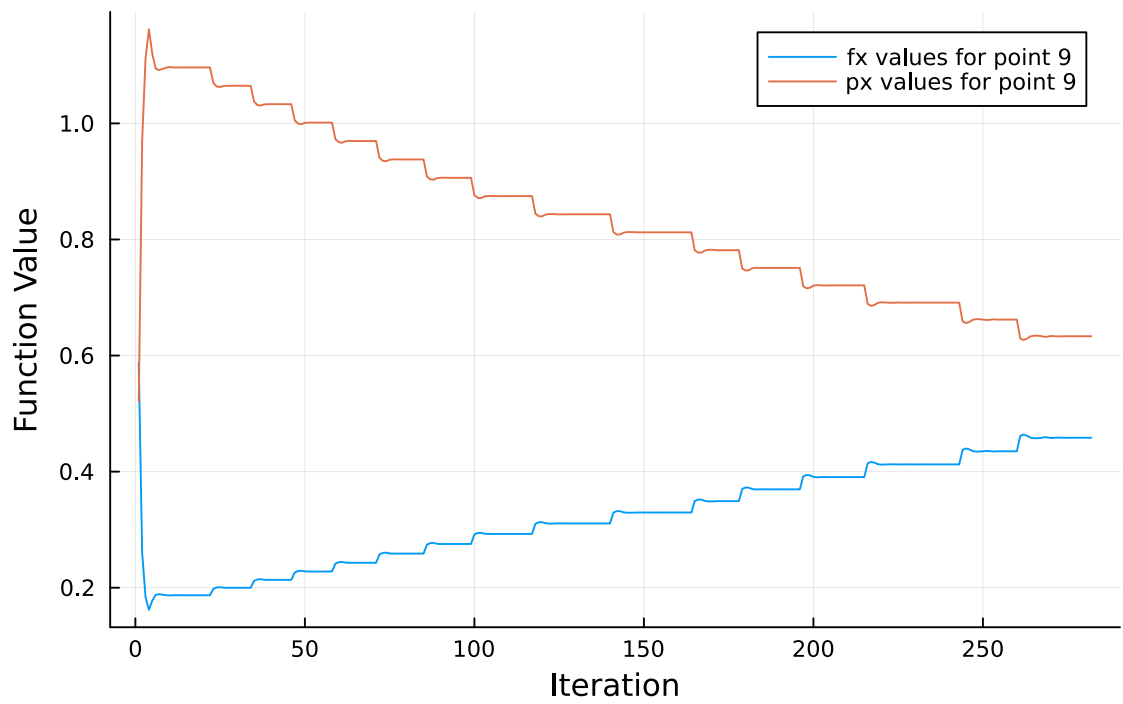
Optimization from Point 7



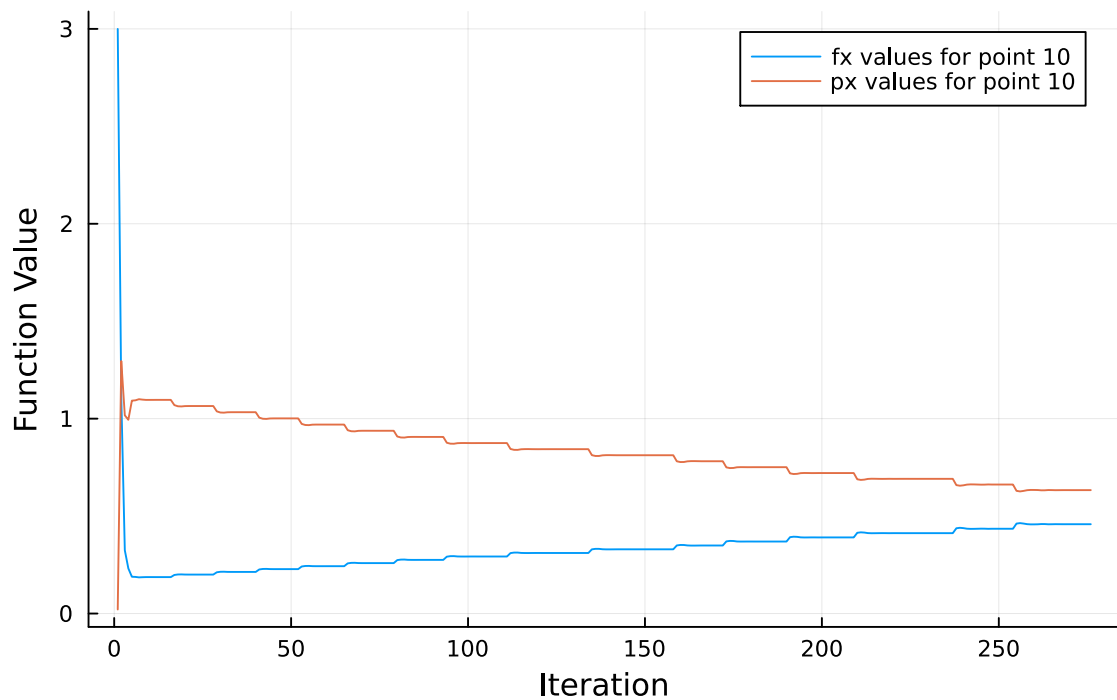
Optimization from Point 8



Optimization from Point 9



Optimization from Point 10



1.9 Optymalizacja metody + benchmarking

W celu przyspieszenia działania funkcji, użyliśmy silnej typizacji oraz zmiany niektórych operacji na "in-place"

Na przykład:

```
f_values = Vector{Float64}(undef, total_iter)
p_values = Vector{Float64}(undef, total_iter)
```

Zamiast

```
f_values = []
p_values = []
```

Oraz

```
@inbounds x .+= α .* d
```

Zamiast

```
x .= x .+ α .* d
```

```
In [ ]: function armijo_line_search_optimized(f, ∇f, x::Vector{Float64}, d::Vector{Float64},
      α0::Float64 = 1.0, c::Float64 = 1e-4, τ::Float64 = 0.0)
      α = α0
```

```

fx = f(x)
gx = ∇f(x)
slope = dot(gx, d)
temp = similar(x)

while true
    @. temp = x + α * d
    if f(temp) <= fx + c * α * slope
        break
    end
    α *= τ
    if α < 1e-14
        return 1e-14
    end
end
return α
end

function penalty_ncg_method_optimized(f, f_grad, p, p_grad, fp, fp_grad,
                                       x0::Vector{Float64};
                                       k_max::Int = 10,
                                       ρ::Float64 = 1.0,
                                       γ::Float64 = 2.0,
                                       atol::Float64 = 1e-8,
                                       max_iter::Int = 50,
                                       tol::Float64 = 1e-6)

    x = copy(x0)

    total_iter = k_max * max_iter
    f_values = Vector{Float64}(undef, total_iter)
    p_values = Vector{Float64}(undef, total_iter)
    idx = 0

    for k in 1:k_max
        r = -fp_grad(x, ρ)
        d = copy(r)
        for i in 1:max_iter
            nr = norm(r)
            if nr < tol
                break
            end

            α = armijo_line_search_optimized(xx -> fp(xx, ρ), xx -> fp_grad(xx,
                                                                    α0 = 0.3, c = 1e-4, τ = 0.5))

            @inbounds x .+= α .* d

            idx += 1
            f_values[idx] = f(x)
            p_values[idx] = p(x)

            r_new = -fp_grad(x, ρ)
            β = (norm(r_new)^2) / (nr^2)

            if (i % 10) == 0
                d = copy(r_new)
            else
                @inbounds d .= r_new .+ β .* d
            end
        end
    end
end

```

```

        r = r_new
    end

    if p(x) < atol
        return x, view(f_values, 1:idx), view(p_values, 1:idx)
    end

    p *= γ
end

return x, view(f_values, 1:idx), view(p_values, 1:idx)
end

```

penalty_ncg_method_optimized (generic function with 1 method)

In [128... `using BenchmarkTools`

In [129... `x0 = [1.0, 1.0]`

2-element Vector{Float64}:

```

1.0
1.0

```

In [130... `@btime penalty_ncg_method_optimized($f, $f_grad, $p, $p_grad, $fp, $fp_grad, $x0, k_max=10, p=1.0, γ=2.0, atol=1e-8, max_iter=100, tol=1`

```

359.800 μs (19521 allocations: 543.86 KiB)
([0.15739590518553295, -1.5219546109917588], [2.0918847512681666, 6.9904460068479
43, 1.0250628011109297, 0.7033760798723971, 0.4679412886613755, 0.489298529554058
2, 0.5585335823359651, 0.5551916740815982, 0.5531032985429739, 0.552617862530357
... 2.125701495493016, 2.1257015203080725, 2.1257015200203764, 2.1257014643769847,
2.125701488442877, 2.125701475246884, 2.125701481023146, 2.125701462086691, 2.125
701481924141, 2.1257014791178745], [8.27655361, 0.0011620270165690414, 0.39243685
18388534, 0.40268732034602134, 0.625821383022456, 0.5984510045550138, 0.524303599
4919741, 0.5273476482038995, 0.5294245276533218, 0.5299095121033472 ... 7.9459395
89925674e-6, 7.945891122977939e-6, 7.945891684833107e-6, 7.946000363139914e-6, 7.
945953359432678e-6, 7.945979132839537e-6, 7.945967851074363e-6, 7.946004836309762
e-6, 7.945966091281398e-6, 7.945971572269842e-6])

```

In [131... `@btime penalty_ncg_method(f, f_grad, p, p_grad, fp, fp_grad, x0; k_max=10, p=1.0, γ=2.0, atol=1e-8, max_iter=100, tol=1`

```

368.500 μs (19522 allocations: 543.95 KiB)
([0.15739590518553295, -1.5219546109917588], [2.0918847512681666, 6.9904460068479
43, 1.0250628011109297, 0.7033760798723971, 0.4679412886613755, 0.489298529554058
2, 0.5585335823359651, 0.5551916740815982, 0.5531032985429739, 0.552617862530357
... 2.125701495493016, 2.1257015203080725, 2.1257015200203764, 2.1257014643769847,
2.125701488442877, 2.125701475246884, 2.125701481023146, 2.125701462086691, 2.125
701481924141, 2.1257014791178745], [8.27655361, 0.0011620270165690414, 0.39243685
18388534, 0.40268732034602134, 0.625821383022456, 0.5984510045550138, 0.524303599
4919741, 0.5273476482038995, 0.5294245276533218, 0.5299095121033472 ... 7.9459395
89925674e-6, 7.945891122977939e-6, 7.945891684833107e-6, 7.946000363139914e-6, 7.
945953359432678e-6, 7.945979132839537e-6, 7.945967851074363e-6, 7.946004836309762
e-6, 7.945966091281398e-6, 7.945971572269842e-6])

```

In []: `for i in 1:length(points)`
`x0 = points[i]`
`t = @btime penalty_ncg_method($f, $f_grad, $p, $p_grad, $fp, $fp_grad, $x0;`
`k_max=10, p=1.0, γ=2.0, atol=1`
`end`


```

397.300 µs (21618 allocations: 597.30 KiB)
354.600 µs (19429 allocations: 542.42 KiB)
394.800 µs (21444 allocations: 593.33 KiB)
417.300 µs (22250 allocations: 609.66 KiB)
366.300 µs (19570 allocations: 545.25 KiB)
380.900 µs (20522 allocations: 572.64 KiB)
395.300 µs (21170 allocations: 587.80 KiB)
393.300 µs (21086 allocations: 583.97 KiB)
351.200 µs (18919 allocations: 528.16 KiB)
358.500 µs (19209 allocations: 533.97 KiB)

```

In [137...

```

for i in 1:length(points)
    x0 = points[i]
    t = @btime penalty_ncg_method_optimized($f, $f_grad, $p, $p_grad, $fp, $fp_g
                                             k_max=10, ρ=1.0, γ=2.0, atol=1
end

```

```

393.400 µs (21618 allocations: 597.30 KiB)
351.200 µs (19429 allocations: 542.42 KiB)
390.500 µs (21444 allocations: 593.33 KiB)
396.900 µs (22250 allocations: 609.66 KiB)
351.800 µs (19570 allocations: 545.25 KiB)
363.500 µs (20522 allocations: 572.64 KiB)
374.600 µs (21170 allocations: 587.80 KiB)
376.000 µs (21086 allocations: 583.97 KiB)
335.300 µs (18919 allocations: 528.16 KiB)
342.000 µs (19209 allocations: 533.97 KiB)

```